Synchronisation Wand

Yuxuan Han

3rd Year Project Final Report

Department of Electronic &
Electrical Engineering

UCL

Supervisor: Thomas Gilbert

April 9, 2025

I have read and understood UCL's and the Department's statements and guidelines concerning plagiarism.

I declare that all material described in this report is my own work except where explicitly and individually indicated in the text. This includes ideas described in the text, figures and computer programs.

I acknowledge the use of Chat-GPT-4o (Publisher OpenAI, URL: https://chat.openai.com) and DeepSeek R1 (Publisher Deepseek, URL: https://www.deepseek.com/) to assist the LaTeX document formatting, grammar checking and content polishing of the report.

This report contains 58 pages (excluding this page and the appendices) and 11945 words.

Signed: Yuxuan Han Date: 09/04/2025

(Student)

Synchronisation Wand

Yuxuan Han

Contents

1 Abstract

Precise synchronisation between wearable inertial measurement units is essential for multi-sensor systems. This work applies a recently proposed encoded electromagnetic pulse (EMP) synchronisation method, which improves timing precision by using customisable pulse sequences to achieve sub-sample-level accuracy. An embedded hardware system, the Synchronisation Wand, was developed based on this method. Building on a prior conference paper that introduced and validated the first version, this work enhances the device with a Bluetooth Low Energy (BLE) interface, achieving 96.38% for both pairing and configuration success rate, RSSI > -80 dBm at 10 m, and 0 crashes in 100 trials. Furthermore, to address the subjectivity and lack of repeatability in previous approaches for validating the encoded EMP synchronisation method, this project proposes a novel, systematic verification platform. A fully automated FPGA-based testbed was developed, offering sub-microsecond sampling trigger control, precise timestamping, and EMP pulse generation accurate to $< 0.01\%$ of target width. The platform also includes an automated data processing component to extract synchronisation patterns from magnetometer data. Together, these enable reliable verification down to $\geq$0.5 ms resolution and set a new standard for the systematic evaluation of synchronisation methods in wearable systems. Using this setup, the encoded EMP method was verified to achieve 0.5 ms accuracy—a 6× improvement over prior results, sufficient for high accuracy requirement applications like gait analysis (1 ms). This work also discusses the method's limitations in scenarios where direct access to magnetometer data is not available.

2 Introduction

2.1 Motivation

Many devices in the domains of ubiquitous and wearable computing include onboard inertial measurement unit sensors (IMUs). Applications of IMUs stretch from the wider topics of human activity recognition [1] and multi-sensor fusion [2] to studies measuring social interaction and engagement in real-world settings [3]. Many applications require precise synchronisation between separate IMU devices, which is an issue that becomes increasingly challenging over extended periods. Most commercial IMUs rely on onboard real-time clocks (RTCs) for synchronization. However, RTCs tend to drift over time at different speed, so that over a long experiment period (e.g. collecting and analysing physical and physiological data for a period over several weeks [4]) their time will vary greatly and introduce large inaccuracy. This is usually due to the inaccurate frequency of the low speed crystal oscillator with causes including manufacture variation and temperature change. To address this issue, several synchronization methods have been developed. These include using kinetic events [5] and simple electromagnetic pulses (EMP) [6]:

- **Kinetic Event Method**: The kinetic event method is the most commonly used wearable synchronisation method among researchers. Performing a kinetic event synchronisation method requires the experimenter or the participant to perform a predefined movement, such as clapping, hitting the table [5, 7, 8], or even tapping the ear [9]. Ideally, this can create similar pulses on the accelerator data of the experimenter's IMUs, which can be aligned accordingly to achieve time synchronisation (See **Figure 1**). However, due to the independent motions between different IMUs, this can cause "ambiguity issues", which means the received kinetic pulses might be different in shapes and hard to align, thus reducing its synchronisation accuracy.
- **Simple EMP Method**: The simple EMP method was introduced recently [6]. It uses the magnetometer and works by aligning the edges of received electromagnetic (EM) events, generated by inductors (See **Figure 1**). Since the EM pulses have clear and sharp edges, this method eliminates the ambiguity issues related to the kinetic event method. However, one limitation of this method is the sampling frequency of the magnetometer. Magnetometers in commercial 9-axis IMUs usually come with a much lower maximum sampling frequency compared to gyroscopes and accelerometers. Using a higher magnetometer sampling frequency also means lower battery life and storage duration, which is sometimes not preferable.

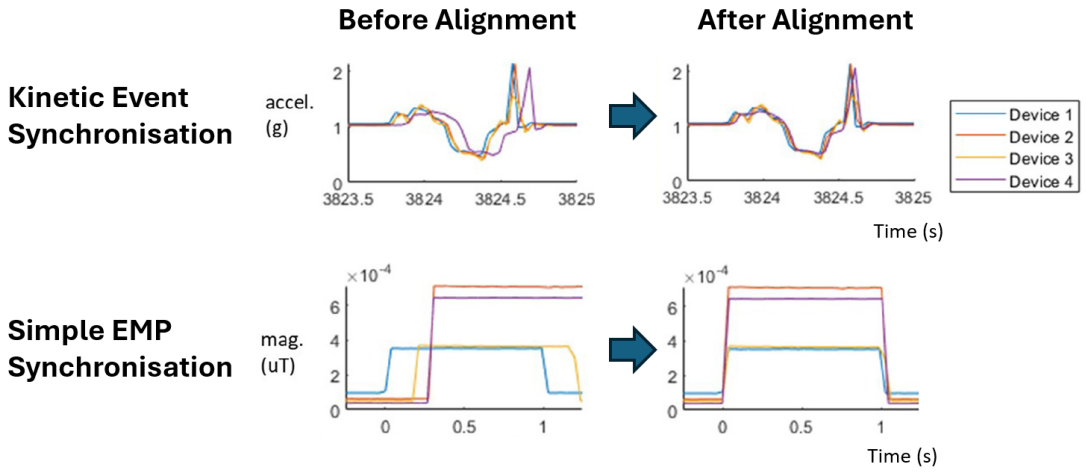


Figure 1: A demonstration of how kinetic event synchronisation is performed by aligning kinetic pulses (upper two graphs), and how simple EMP synchronisation is performed by aligning the rising edges of the EM pulses (lower two graphs). (Source: Figure 4, [10])

Recently, a new approach using encoded electromagnetic pulses has been developed, which eliminates the ambiguity issue as well and is able to expand the precision to be finer than the magnetometer sampling period [10]. Details of this method will be introduced in **Section 3.1**. This new method has high application value but currently no pre-existing hardware implementation to perform it.

Additionally, the previous work of the encoded EMP synchronisation method still have one aspect that require further improvement: The previous verification approach [10] was noted to be insufficiently systematic, relying heavily on manual interpretation and lacking repeatability, according to the journal editor's feedback. This introduces potential human bias and limits the robustness of the evaluation. Therefore, a more structured, repeatable and precise verification methodology and setup is required to strengthen the validation of the method.

2.2 Goal, Objectives and Metrics

The main goal of this third-year project can be divided into three key components:

1. Develop an embedded hardware solution for executing encoded EMP synchronisation (the *Synchronisation Wand*).
2. Design a systematic verification platform for evaluating the encoded EMP synchronisation method, including both hardware testbed and data processing scripts.
3. Conduct further verification and analysis of the encoded EMP synchronisation technique under varying conditions.

The objectives and corresponding evaluation metrics for each component are outlined below:

The Synchronisation Wand:

1. **Design and implement the hardware and firmware** required to perform encoded EMP synchronisation. The system should achieve a synchronisation error no greater than the theoretical maximum defined by the encoded electromagnetic pulse method [10].
2. **Integrate Bluetooth Low Energy (BLE) functionality** to support real-time device configuration via a mobile interface. The BLE module should maintain a stable connection with an average received signal strength indicator (RSSI) no lower than -80 dBm at a range of 10 metres in line-of-sight conditions—commonly used as the threshold for poor BLE reception [11]. Pairing success rate must be $\geq 95\%$, and configuration commands sent via BLE must have success rate $\geq 95\%$ (a value recommended by Google for good user interface [12]), with no system crashes.

It is worth noting that a conference paper documenting the first version of the *Synchronisation Wand* has already been published [13], with the author of this report listed as first author. That work details the initial design and validation. As such, this report focuses only on the new developments—specifically the integration of BLE functionality.

Systematic Verification Platform Design:

1. **Develop a hardware testbed** to systematically verify the performance of encoded EMP synchronisation. The testbed must generate synchronisation pulses with a timing error below 0.01% of the pulse width. It should also enable magnetometer sampling with a maximum timing error of 0.5ms to support synchronisation accuracy verification at the 1ms level—a commonly used value for high accuracy applications like gait analysis [14]. This 2:1 relationship follows the principle behind the Nyquist sampling theorem.
2. **Implement an automated testing mechanism** within the testbed. It must be capable of running over 100 successive synchronisation experiments without human intervention and main-

tain a 0% crash rate. The system must also support automatic adjustment of test parameters (e.g., T_{OFFSET}) between test cycles.

3. **Develop a pre-processing program** capable of automatically detecting, classifying, and annotating encoded EMP synchronisation pulses from raw magnetometer data. The program should demonstrate robustness and achieve at least 95% accuracy when evaluating 100 synchronisation patterns.

Further Verification of the Encoded EMP Synchronisation:

1. **Conduct verification of the encoded EMP synchronisation method** across five different accuracy levels, including one that exceeds the resolution required for gait analysis applications (1 ms) [14].

2.3 Structure of this Report

This report is structured into three main sections—Technical Background, Methodology, and Results and Discussion—to clearly present the development and evaluation of the Synchronisation Wand and its verification system.

- The **Technical Background** section introduces the key concepts and technologies underpinning the project. It explains the encoded EMP synchronisation method, core design elements of the Synchronisation Wand, BLE implementation on microcontrollers, limitations of previous verification methods, the role of FPGA-based testing, timing-critical magnetometer configurations, and the statistical method used for binary test analysis.
- The **Methodology** section outlines the implementation and verification process, including:
    1. BLE integration and testing on the Synchronisation Wand.
    2. FPGA testbed design and timing verification.
    3. Analysis of magnetometer measurement time lag.
    4. Development of a sample pattern pre-processing program.
    5. Experimental setup for testing encoded EMP synchronisation method performance on two types of magnetometers.
- The **Results and Discussion** section presents outcomes from the above methods, including:
    1. Performance of BLE.
    2. Testbed timing accuracy and automation reliability.
    3. Impact of sensor time lag variation to the whole system.
    4. Verified reliability of the encode EMP synchronisation method.
    5. Limitations of magnetometers without direct host access.

3 Technical Background

3.1 The Encoded EMP Synchronisation Method

The encoded electromagnetic pulse (EMP) synchronisation method, proposed by Gilbert et al. in *"A magnetometer-based method for in-situ syncing of wearable inertial measurement units"* [10], leverages sequences of encoded EMP signals to reduce the theoretical maximum synchronisation error below the magnetometer sample period.

3.1.1 Implementation Workflow

A typical synchronisation process involves aligning the RTC of IMU devices to a low-drift master device that generates encoded EMP pulses. The master device maintains a precise local time reference, which can be synchronised to Coordinated Universal Time (UTC) via protocols like NTP or GPS. During an experiment, the user triggers an encoded EMP signal from the master device toward a target IMU, while the master records a reference timestamp marking the start of the synchronisation event. Synchronisation can then be done with post-experiment data analysis (e.g., in MATLAB) with two stages:

1. **Rough Time Adjustment**: Large desynchronisation offsets (exceeding the magnetometer sample period) are corrected by aligning the first rising edge of the EMP signal in the IMU's magnetometer data with the master's reference timestamp. This step resembles the alignment process used in the simple EMP method.

2. **Fine Time Adjustment**: Residual sub-magnetometer-sample-period offsets are resolved using the encoded EMP sequence. By analysing the variable-width pulse pattern (e.g., an initial pulse followed by incremental extensions), the exact timing discrepancy within the sample period is calculated and corrected.

After synchronisation, the IMU's RTC is calibrated to match the master device's reference clock. For multi-device experiments, repeated synchronisation events align all IMUs to the master's common timebase, ensuring temporal consistency across recordings.

To mitigate long-term RTC drift, experimenters can perform periodic encoded EMP synchronisation events (e.g., at the start, middle, and end of an experiment). By interpolating timestamps between these events and stretching the data timeline to match the master's drift profile, all IMU recordings inherit the master's stable time reference, enabling precise cross-device data fusion.

3.1.2 Theory

To perform this synchronisation, the master device will send a series of successive electromagnetic pulses, as shown in **Figure 2**.

The series of pulses contains an initial pulse with width w and several successive pulses with width $w + a$, where a is the shift amount and also determines the synchronisation resolution (maximum synchronisation error). w is constrained to be an integer multiple of the target magnetometer sample period s. Thus, for the initial pulse, unless for a perfectly synchronised case where m = 0, the normal number of samples allowed inside the pulse is defined by w/s, which is 2 in **Figure 2**. For every successive pulse, this shift amount reduce the value of m by a, until it is small enough to allow an extra sample in the pulse (See red lines in **Figure 2**). By locating the corresponding extra-sample-pulse index (p), the time offset (m) be estimated within the range specified by **Eq. 1**, and time calibration can be done by the user accordingly.

$$(p-1)a < m \leq pa \tag{1}$$

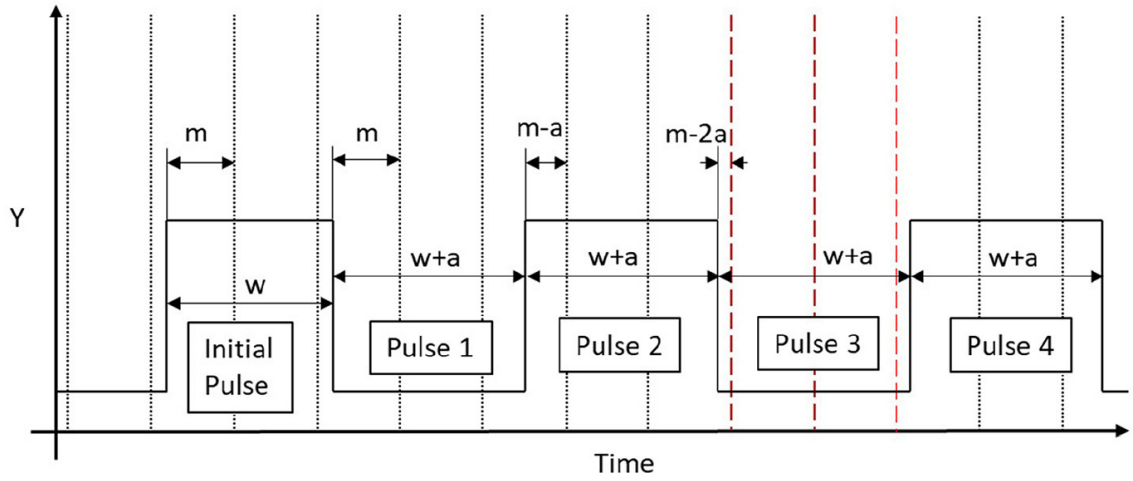


Figure 2: A graphical representation of the encoded EMP synchronisation method demonstrating how the distance, m, between the starting edge of each pulse and the next sample reduced by the extended width, a. The vertical lines represent magnetometer samples. Note that pulse 1 and 2 has only two samples between edges while pulse 3 has an additional sample pulse. (Source: Figure 6, [10])

3.1.3 Synchronisation Signal Customisation

This synchronisation signal is highly customisable. The user need to configure it appropriately before using, depending on their hardware and their need. Three of the most important parameters are:

- **Target Magnetometer Sample Period** (s), which will affect the value selection of pulse width.
- **Initial Pulse Width** (w), which should be an integer multiple of s and should be greater than the transient response time of the electromagnet.
- **Maximum Synchronisation Error** (a). Any real positive number accepted, adjust according to the need of the experiment. Lower value means more accurate synchronisation but longer synchronisation time required.

Based on these configurations, the minimum number of pulses required is $s/a + 1$, to ensure a guaranteed extra sample in the EMP sequence.

3.2 The Synchronisation Wand Fundamentals

3.2.1 Microcontrollers

A microcontroller is an integrated system-on-chip that combines processing, memory, and programmable I/O to execute dedicated tasks in embedded applications. In the Synchronisation Wand, it performs precise electromagnetic pulse (EMP) generation, manages real-time clock synchronization, and interfaces with peripherals (e.g., OLED, SD card). Compared to general-purpose processors or FPGAs, microcontrollers provide optimal balance of real-time control, energy efficiency, and cost-effectiveness for portable devices [15]. The Synchronisation Wand is based on an ESP32-S3 microcontroller from *Espressif*, which is selected for its dual-core 240 MHz processor (enabling multitasking), integrated Wi-Fi/BLE (for system time synchronisation), low-power operation, wide peripheral support and low cost [16].

3.2.2 Inertial Measurement Unit (IMU)

An Inertial Measurement Unit (IMU) is a widely used electronic component that combines sensors like accelerometers, gyroscopes, and sometimes magnetometers to measure linear acceleration, angular rotation, and magnetic field direction. A 9-axis ICM20948 IMU is installed on the synchronisation

wand to add additional support to the kinetic event synchronisation method.

3.2.3 Temperature Compensated Crystal Oscillator (TCXO)

A Temperature-Compensated Crystal Oscillator (TCXO) is a precision clock source designed to minimize frequency drift caused by temperature changes in standard crystal oscillators (XOs). The synchronisation wand uses a 3.8 ppm 32.768 kHz ATXK-H11 TCXO from *ABRACON* [17], providing low-drift local reference time for IMU device synchronisation.

3.2.4 Simple Network Time Protocol (SNTP)

Precise synchronisation wand system time update is achieved by the Simple Network Time Protocol (SNTP), which is natively supported by ESP-IDF. SNTP is a streamlined version of the Network Time Protocol (NTP), designed to synchronize clocks across networked devices with minimal complexity for resource-constraint systems. The accuracy of SNTP time synchronisation is typically around 10-100 ms, subject to network instability [18].

3.2.5 Printed Circuit Board (PCB) Stack Up and Controlled Impedance Traces

Printed Circuit Board (PCB) stack-up refers to the arrangement of layers in a multi-layer PCB. A common stack-up is Signal-Ground-Ground-Signal for 4-layer PCBs, which is also used by the synchronisation wand due to its good electromagnetic compatibility for mixed signal systems by ensuring stable reference ground planes for all signal return paths [19].

Some critical signals require their traces' characteristic impedance to be controlled. For example, in the synchronisation wand design, 90Ω traces are used for USB differential pair [20], and 50Ω traces are used for antenna feeding to maximum RF efficiency. These values are achieved by using appropriate PCB waveguide type and precisely calculating trace geometry (width, spacing) and dielectric properties (substrate permittivity, layer thickness).

3.2.6 ESP-IDF and FreeRTOS

ESP-IDF is the official development framework for Espressif's SoCs. It is used for programming the firmware of the synchronisation wand. Compared to Arduino (which simplifies development with abstraction and limited hardware control), ESP-IDF offers deeper hardware access, optimized performance for complex tasks, native support for dual-core processing, and robust real-time operation system (RTOS) integration [21]. This makes it better suited for customising real-time, resource-intensive embedded systems requiring precise timing like the synchronisation wand.

An RTOS manages hardware resources and ensures deterministic task execution within strict timing constraints. The synchronisation wand's firmware uses FreeRTOS, a lightweight open-source RTOS that provides task scheduling, inter-task communication (e.g., queues, semaphores), and memory management [22]. It enhances responsiveness by enabling concurrent task execution, prioritizes critical operations (e.g., sensor data processing), and optimizes resource usage through preemptive multitasking.

3.2.7 Interrupts

In microcontrollers, an interrupt is a hardware-triggered mechanism that temporarily suspends the main program flow to execute a specific Interrupt Service Routine in response to an event. This enables real-time responsiveness without constant polling, which wastes computational resources. Commonly used interrupt types include the general purpose input / output (GPIO) interrupt, which is triggered by external signal change (used for push buttons of the synchronisation wand), and timer interrupts, triggered when internal hardware timer counter value increase and reach a pre-defined amount (See **Figure 3**). Timer interrupts are good for time-critical tasks due to its high accuracy, including the synchronisation signal generation in the synchronisation wand.

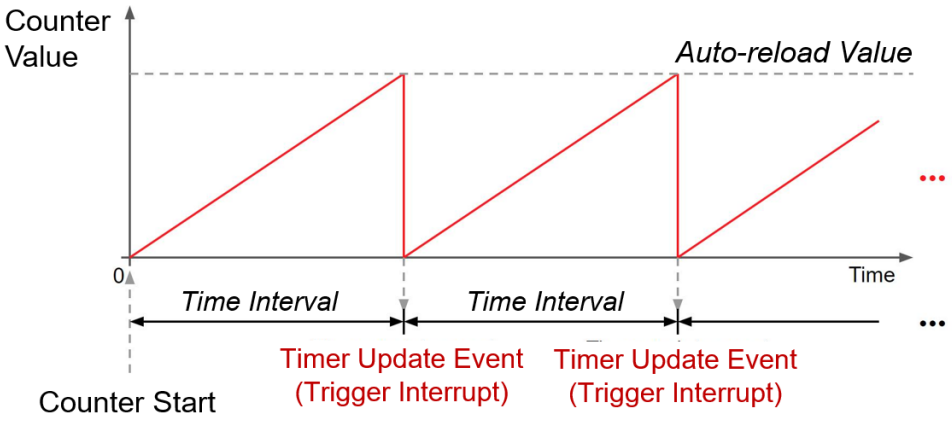


Figure 3: A demonstration of the timer interrupt. Note that the counter value increases with time, and when it reaches the pre-defined auto-reload value, it will be reset and trigger an interrupt.

3.2.8 Bluetooth Low Energy (BLE)

Bluetooth low energy (BLE) is the major feature update in the new version of synchronisation wand, which is used to enable synchronisation parameter adjustment via mobile phone app. Its standard is being defined by the Bluetooth Special Interest Group [23]. BLE is a short-range 2.4 GHz wireless communication protocol optimised for low power data exchange. Unlike Classic Bluetooth (designed for continuous data streaming, e.g., audio), BLE minimizes energy use through short-burst communication and deep sleep modes (Approximately 1% to 5% of classic Bluetooth [24]). Additional advantages include rapid pairing (<3ms), adaptive connection intervals (7.5ms–4s) for balancing latency/power [25], and a standardized service framework (e.g., heart rate monitoring, battery level) for seamless interoperability.

BLE Protocol Stack Overview:

The typical BLE protocol stack structure is shown in **Figure 4** below, which can mainly be divided into three parts: application, host and controller.

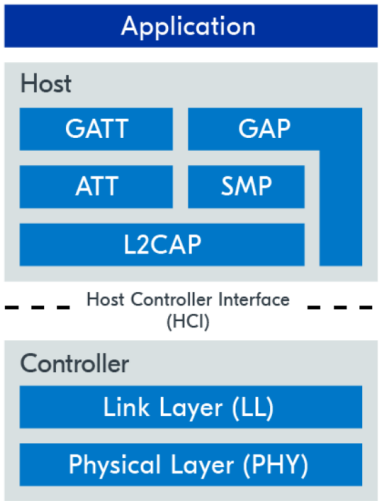


Figure 4: The bluetooth low energy (BLE) protocol stack structure. The firmware of the synchronisation wand is built on the top application layer. When using BLE, the firmware need to communicate with the host through application programming interface (API), and the host will handle remaining parts. (Source: [26])

The user firmware will be programmed on the top application layer, and interact with the host through

the provided application programming interface (API). The important part of the application layer include profiles, services and characteristics. The host determines how BLE devices store and exchange data between each other, and at the lowest layers, the controller containing the physical layer is responsible for the generation, reception of the radio signal and encode and decode of the data.

For a detailed explanation of the layers contained inside the host:

- **Logical Link Control & Adaptation Protocol (L2CAP)**: provides data encapsulation services to the upper layers.
- **Security Manager Protocol (SMP)**: defines and provides methods for secure communication.
- **Attribute Protocol (ATT)**: allows a device to expose certain pieces of data to another device.
- **Generic Attribute Profile (GATT)**: defines the necessary sub-procedures for using the ATT layer.
- **Generic Access Profile (GAP)**: interfaces directly with the application to handle device discovery and connection-related services.

GAP - Device Roles and Topologies:

The Generic Access Profile (GAP) defines complementary role pairs to enable flexible BLE communication architectures:

1. **Connection Oriented Communication - *Peripheral and Central***:
   A *Peripheral* (e.g., sensor) operates as a low-power endpoint that advertises its availability through periodic broadcast packets. It establishes bidirectional connections only when requested by a *Central* device (e.g., smartphone), which actively scans for peripherals and manages all initiated links. This asymmetric relationship forms point-to-point or star topologies **See Figure 44**, where the central coordinates data aggregation while peripherals optimize for energy efficiency (<1 A sleep currents).
2. **Broadcast Communication - *Broadcaster and Observer***:
   A *Broadcaster* (e.g., beacon) transmits non-connectable advertising packets in one-way communication mode, prioritizing ultra-low power consumption through intermittent transmissions. Correspondingly, an *Observer* passively scans and decodes these broadcasts without establishing connections. This pair enables connectionless topologies (See Figure 44) for scenarios requiring minimal power overhead, such as location tracking or environmental sensing systems.

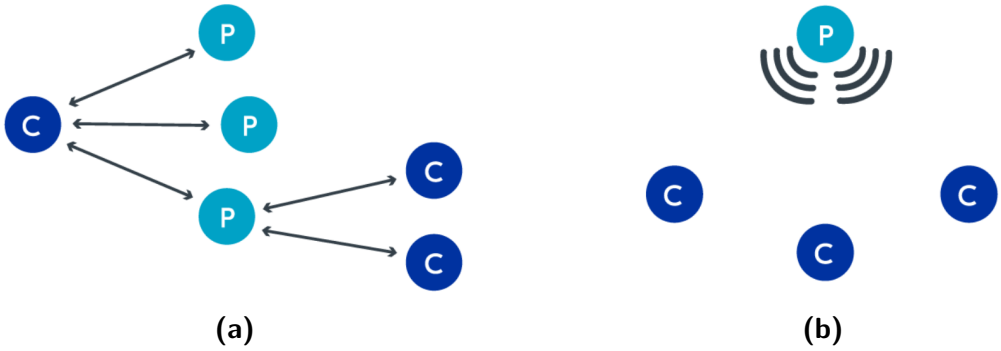


Figure 5: Two different bluetooth low energy (BLE) topologies with "C" representing Central / Observer and "P" representing Peripheral / Broadcaster. (a) Connected topology, point-to-point or star. (b) Broadcast topology. Note that the synchronisation wand acts as a peripheral inside a connected topology. (Source: [26])

For the BLE application scenario in the synchronisation wand, the wand will act as the peripheral

and the mobile phone acts as central. The topology used will be point-to-point connection type. To perform data exchange via connection, ATT and GATT plays an important role.

ATT & GATT - Data Representation and Exchange:

Bluetooth Low Energy (BLE) employs the Attribute Protocol (ATT) and Generic Attribute Profile (GATT) to enable efficient data management. ATT establishes a lightweight client-server communication framework, where the fundamental data unit is the Attribute (See **Figure 6**), defined by four components:

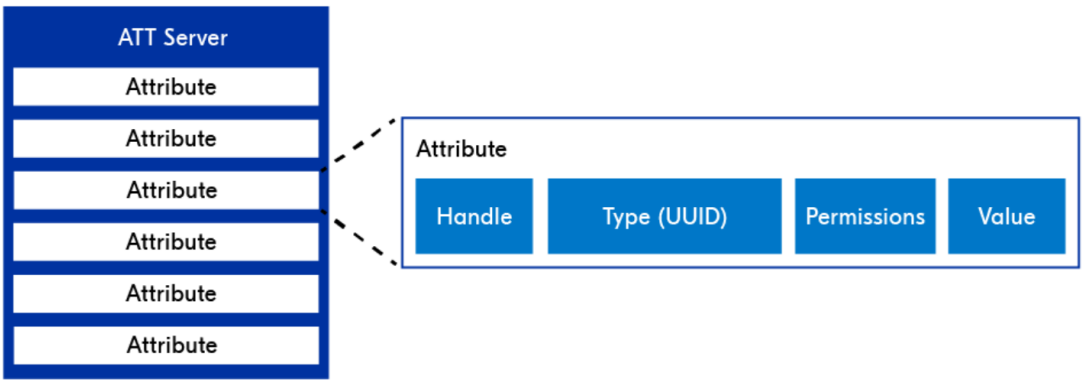


Figure 6: A view of the structure of an Attribute Protocol (ATT) server. The ATT server contains the data that is allowed to be exposed to other devices via BLE. The smaller units contained inside the ATT server are attributes, which have four fields: handle, UUID, permission and value. (Source: [26])

- **UUID**: A 16-bit short UUID (e.g., Device Information Service 0x180A) for Bluetooth SIG-defined data types, or a 128-bit long UUID for developer-customized services.
- **Handle**: A 16-bit unique address for rapid client-side attribute addressing.
- **Permissions**: Access control flags (e.g., read-only for sensor data or writable for configurations).
- **Value**: A binary data block (up to 512 bytes) with compact encoding (e.g., 1-byte Boolean or 4-byte floating-point values).

GATT extends ATT by organising attributes into a hierarchical structure (See **Figure 7**):

- **Service**: A functional module (e.g., Heart Rate Service 0x180D) containing multiple characteristics.
- **Characteristic**: A data point within a service (e.g., Heart Rate Measurement 0x2A37), comprising a value field, property flags (e.g., readable, notifiable), and descriptors.
- **Descriptor**: Metadata for configuration (e.g., Client Characteristic Configuration Descriptor (CCCD) to enable notifications).

Data exchange operates via two modes:

- **Client-initiated requests**: *Read* Request to retrieve values or *Write* Command to update parameters.
- **Server-initiated asynchronous updates**: *Notification* for low-latency unidirectional transmission (e.g., real-time temperature updates) and *Indication* for reliable delivery (requiring client acknowledgment).

The Attribute Table, stored linearly on the server, holds all attributes. Clients parse the table structure through protocol-layer discovery processes (Service Discovery) to establish logical mappings. For

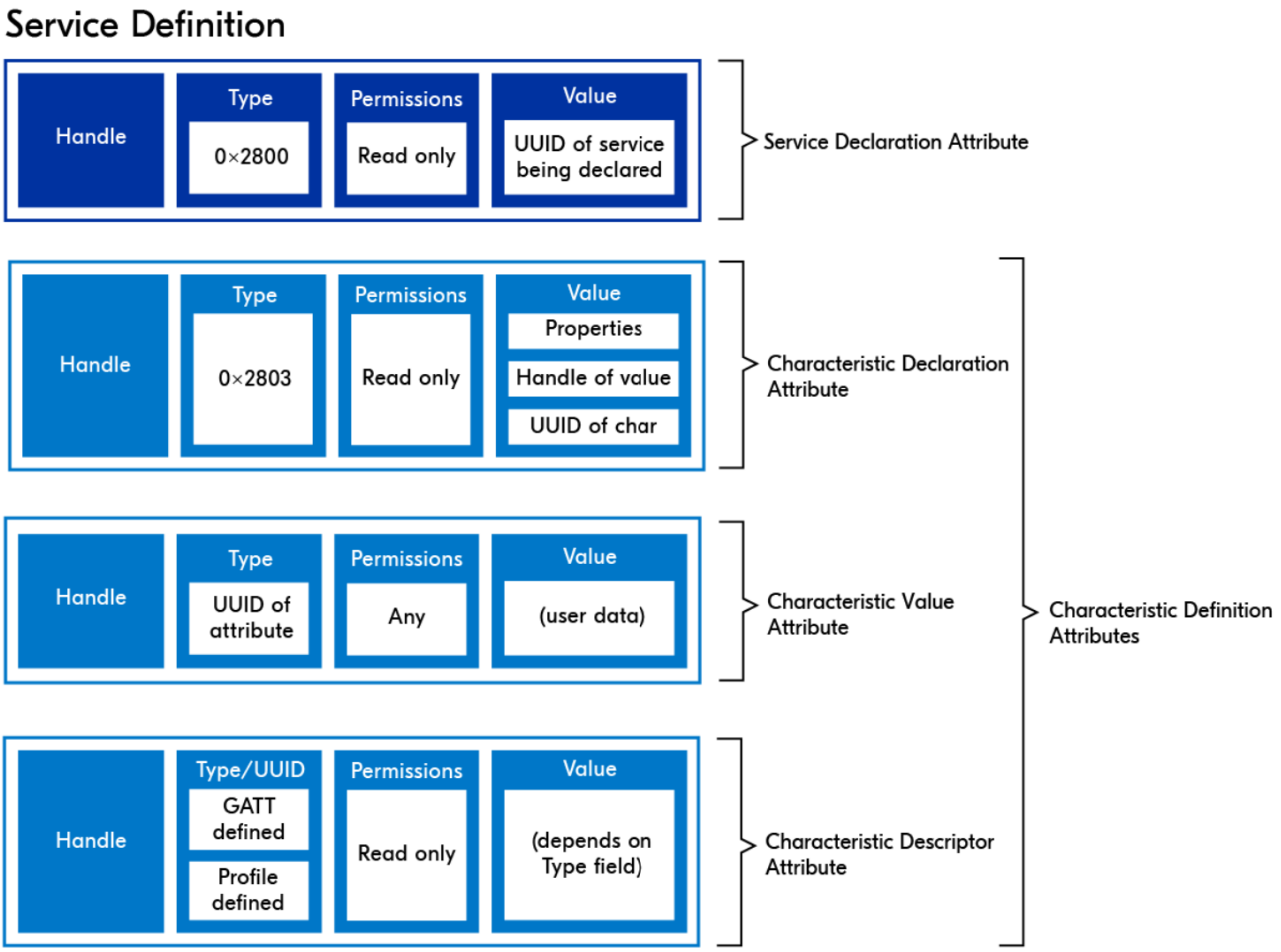


Figure 7: The structure of a Generic Attribute Profile (GATT) service consists of a characteristic that contains a single value and an associated descriptor. This will be the most important structure for BLE implementation in the synchronisation wand. (Source: [26])

instance, the standard Battery Service's attribute table sequentially stores service declarations, characteristic definitions, and CCCD descriptors, with a total memory footprint typically under 1 KB.

3.3 The Field Programmable Gate Array (FPGA) Based Testbed Fundamentals

3.3.1 Previous Verification Method Limitations

The encoded EMP synchronization method is currently validated by performing kinetic events (e.g., table impacts) across multiple IMU devices, followed by manual alignment in MATLAB. In this workflow, the data is first aligned using the encoded EMP synchronisation method, and a secondary "expert opinion" alignment of the kinetic event serves as an idealized ground truth for error calculation. [10] However, this approach presents three major limitations:

- **Subjective Bias in Manual Alignment**: The reliance on human judgment introduces subjectivity, leading to potential inconsistencies in reference timestamps among different operators.
- **Uncertainty in Kinetic Event Signals**: Sensor noise, variations in device response, and environmental factors can obscure the kinetic event signals, resulting in an ambiguous alignment reference.
- **Lack of Repeatability**: The manual nature of the process compromises repeatability across

experimental batches, making it difficult to ensure consistent validation conditions.

These limitations hinder the reliability of sub-millisecond synchronization accuracy assessments.

3.3.2 Role of the FPGA in the Testbed

To overcome the limitations stated in the previous section, the subjective manual alignment should be replaced with an objective ground truth based on controlled time offsets. A testbed can be designed to inject precisely defined timing error into IMU devices (e.g., a fixed +15 ms offset), enabling automated comparison between the injected offsets and the EMP synchronization results.

An FPGA is chosen as the core of the testing platform instead of a conventional microcontroller, based on three key advantages:

- **Parallel Processing for Multi-Device Synchronization**: The FPGA's parallel hardware architecture allows simultaneous emulation of independent time offsets across multiple IMU devices, and perform data acquisition and timestamping. In contrast, microcontrollers (e.g., ESP32) rely on sequential execution within a CPU core, making it difficult to handle multiple precision tasks within the same clock cycle, increasing scheduling-induced errors.
- **Hardware-Level Precise Timing Control**: Hardware level design in FPGAs enables nanosecond-level timestamp resolution, ensuring precise sample timestamping and controlled timing offset injection. Additionally, the processing process in FPGAs are more deterministic, and the signal path from input to timestamp recording follows a fixed delay, eliminating timing uncertainties. With this advantage, FPGAs are commonly used in high precision time and clock synchronisation researches [27]. In contrast, MCU-based timestamping typically relies on software-interrupt-driven timers and RTOS tasks, which are prone to random jitter due to OS scheduling and interrupt latency and have much lower accuracy typically in microsecond level.

The testbed in this project is implemented on a Digilent Nexys Video FPGA development board, which features a Xilinx Artix-7 XC7A200T-1SBG484C FPGA. This mid-range FPGA provides 215,360 logic cells and 6.8 Mb block RAM [28], enabling parallel emulation of 12+ IMU devices with independent drivers and timing profiles. And the design is developed using Xilinx Vivado, a unified hardware/-software co-design environment that enables full-stack FPGA implementation from RTL synthesis to bitstream generation.

3.3.3 Magnetometer Essentials

Magnetometer is the core component to sample the received encoded electromagnetic pulses and perform synchronisation. A magnetometer detects magnetic fields by measuring their vector components (strength and direction) through principles such as the Hall effect. The hall-effect sensor generate a voltage perpendicular to current flow when exposed to a magnetic field, proportional to field strength. This voltage is then amplified, sampled and processed within the magnetometer, finally output in the form of signed integer values. The following are some important configurations that will greatly affect the operation and timing of the magnetometer and need special attention if highly accurate sample timestamp is needed.

Active Measurement Axis: Most magnetometers support magnetic field measurement in all x, y and z directions. These are internally achieved by using different hall sensors by time-multiplexing.

Sample Mode and Sample Frequency: A basic magnetometer has at least two different operation modes: The single-shot mode and continuous measurement mode. At the single shot mode, the magnetometer only perform measurement once when a trigger signal (either through a register write or a physical pin) is received. At continuous measurement mode, the magnetometer perform measurement continuously every specific time interval, which is defined by the sample frequency configured by the user.

Hall Plate Spinning: To mitigate inherent offset errors (e.g., from process variations or temperature drift), spinning current techniques dynamically rotate the Hall plate's biasing and sensing configurations. In 2-phase spinning, current polarity is reversed, averaging out offsets over two cycles. 4-phase spinning cycles biasing and sensing contacts in four orthogonal directions (e.g., 0°, 90°, 180°, 270°), further suppressing residual offsets and low-frequency noise by exploiting symmetry (See **Figure 8**). The 2-phase spinning is faster and suitable for time-sensitive applications, while 4-phase spinning is more accurate and suitable for precise measurement needs.

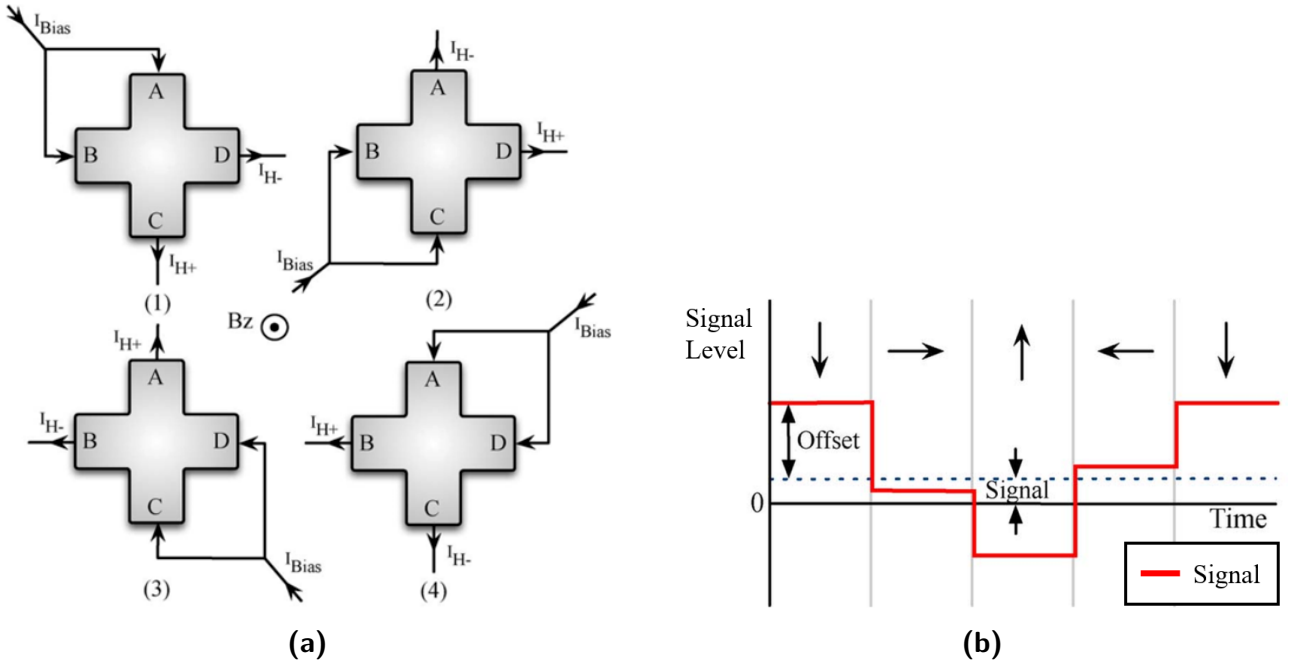


Figure 8: The principle of hall plate spinning technique. (a) bias current applied to the hall plate in four different directions, time multiplexed. (b) How the hall spinning technique is able to mitigate inherent offset errors and obtain the correct signal. (Source: Figure 4. [29])

ADC Oversample Rate: The analogue hall voltage signal is digitized using an ADC (analog-to-digital converter) operating at an oversampled rate (e.g., 10–100× the Nyquist frequency). Oversampling reduces quantization noise and improves SNR (signal-to-noise ratio) by spreading noise across a wider bandwidth. Greater oversampling rate will result in more accurate sample value, but much longer sample time and much greater power consumption.

Digital Low-Pass Filter: The internal digital low-pass filter can be used to reduce high-frequency noise and smooth the readings. However, greater level of filtering will significantly limit the maximum sample frequency and increase the single measurement time due to more sample points and longer processing time needed.

3.3.4 Magnetometer Case Study: MLX90393 and ICM20948 (AK09916)

MLX90393:

MLX90393 is a 3-axis magnetometer chip from *Melexis* [30]. Compared to similar products like LIS3MDL from *STMicroelectronics* [31] and MMC5603 from *Memsic* [32], it is chosen to be the main magnetometer used by the testbed for encoded EMP synchronisation verification due to the following superior unique features:

1. **High customisability**: The MLX90393 magnetometer is highly customisable, including all the features mentioned in **Section 3.3.3**.

2. **Highly detailed datasheet**: The MLX90393 magnetometer has a highly detailed datasheet, especially for the measurement timing part, which is seldom found in the datasheet of similar products. The exact measurement timing values provided can assist more accurate timestamp recording.
3. **Data ready (DRDY) interrupt pin**: The MLX90393 magnetometer has a physical DRDY interrupt pin which generate high signal when the measurement data is ready to be read. This can be directly wired to the FPGA pin to trigger data recording logic and achieve more accurate timing, without the need of constantly polling the DRDY register via SPI.
4. **Ultra fast measurement speed**: The MLX90393 magnetometer is able to achieve a fastest measurement time of only 192 μs, far smaller than other magnetometers (e.g., 7.2 ms for AK09912 from *AKM* [33]). This small measurement time window allows sub-millisecond accurate timestamp recording.

In the test bed implementation, single measurement mode is used instead of continuous measurement mode to achieve a more precise and configurable sampling timing control. The typical operation cycle of this mode is shown in **Figure 9**:

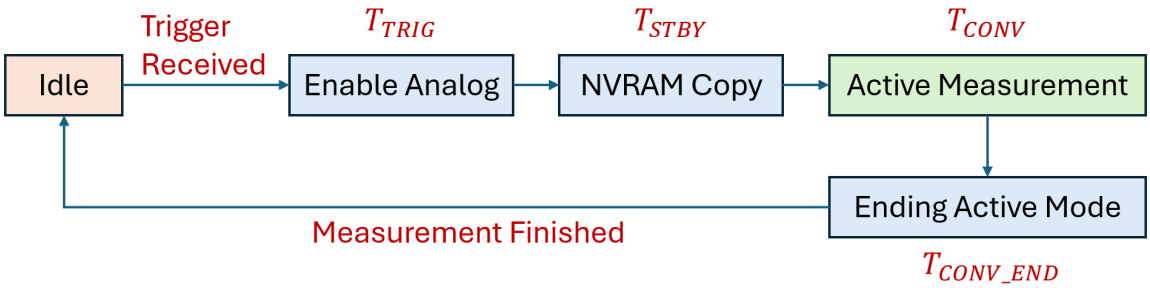


Figure 9: State diagram of the single measurement in MLX90393. It is important to note that the active measurement does not directly happen after measurement trigger is asserted. Instead, it need to pass two internal states with delays T_{TRIG} and T_{STBY}.

ICM20948 (AK09916):

The ICM-20948 is a widely used 9-axis IMU in wearable applications. It integrates an AK09916 3-axis magnetometer, which is physically co-packaged on the same die but functions as an independent I^2C device. As illustrated in **Figure 10**, the AK09916 is accessible only via an internal I^2C bus. This architecture presents a unique characteristic of interest for this project.

When the ICM-20948 is connected to a host system via serial peripheral interface (SPI) (instead of I^2C), the host cannot directly access the magnetometer. Instead, communication must be mediated by the internal I^2C master controller embedded within the ICM-20948. This controller periodically polls the AK09916's registers and transfers the magnetometer data to the ICM-20948's shared sensor register bank, from which the host can then retrieve it via SPI.

This indirect access mechanism introduces potential latency and timing uncertainty. Investigating the extent to which this indirect access affects synchronization accuracy is a key motivation for selecting this chip in this project.

3.3.5 Sensor Communication Protocol: Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous, full-duplex serial communication protocol widely used for high-speed data exchange between microcontrollers, sensors, memory modules, and peripheral devices. Operating in a master-slave configuration, SPI employs four primary signals: SCLK (serial clock, generated by the master), MOSI (master-out-slave-in), MISO (master-in-slave-out), and SS/CS (slave select/chip select). Unlike I^2C, SPI does not require device addressing; instead,

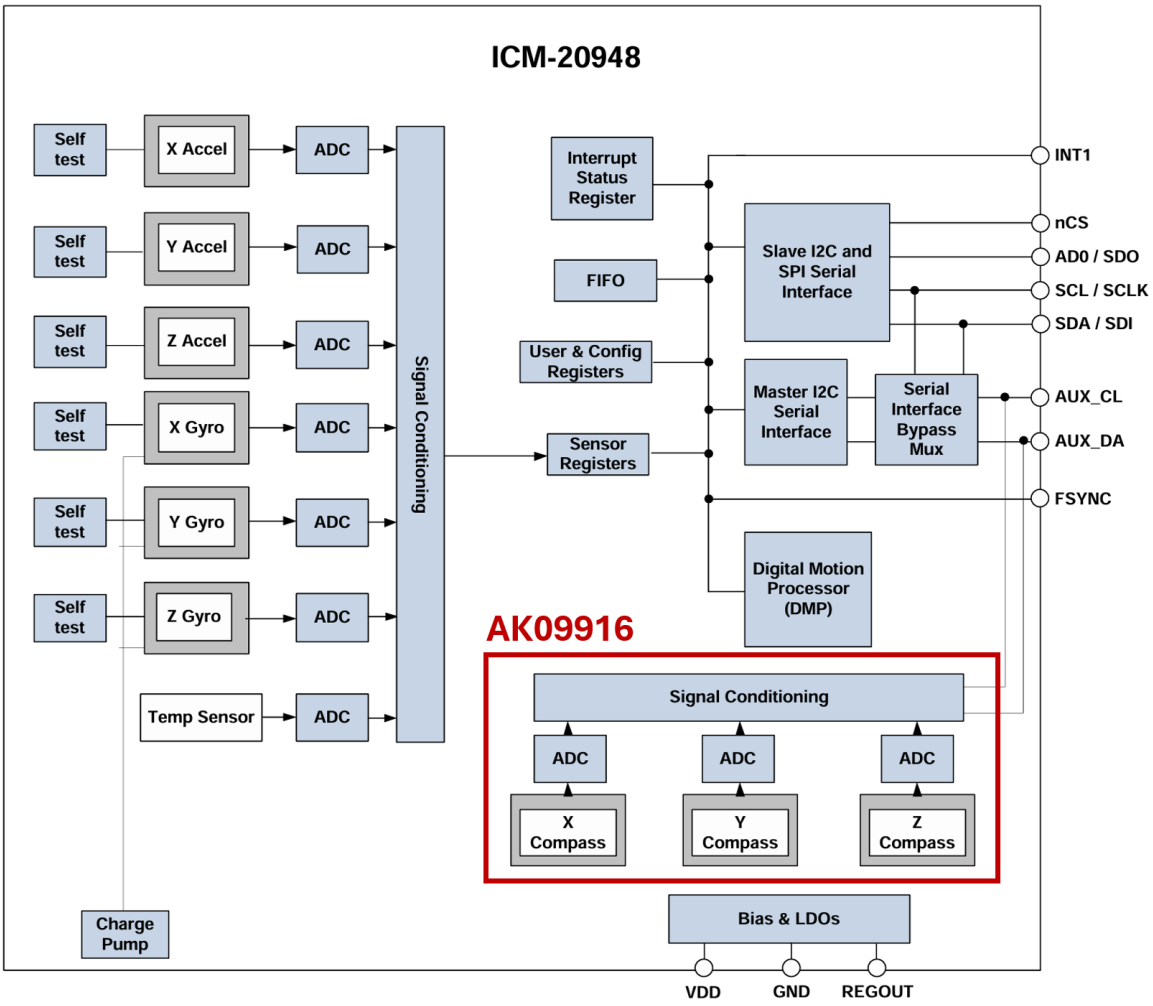


Figure 10: ICM20948 system block diagram. Note that the master device on SDA / SDO lines do not has direct access to the AK09916 magnetometer chip. (Source: [34])

the master selects a slave by pulling its dedicated SS line low, enabling direct point-to-point or daisy-chained communication. SPI has four modes, with different CPOL (clock polarity) and CPHA (clock phase) settings (See **Table 1** and **Figure 11**). The ICM20948 chip works on SPI 0 mode, while the MLX90393 mode work on SPI 3 mode.

Mode	CPOL	CPHA	Clock Starts On	Data Sampling Edge	Data Transition (shifted out) Edge
0	0	0	Low	Leading (Rising)	Trailing (Falling)
1	0	1	Low	Trailing (Falling)	Leading (Rising)
2	1	0	High	Leading (Falling)	Trailing (Rising)
3	1	1	High	Trailing (Rising)	Leading (Falling)

Table Notes:
Leading edge: First clock transition after CS activation
Trailing edge: Second clock transition in same phase

Table 1: SPI Operational Modes with Edge Timing Specification. Note that the ICM20948 chip used in this project works on SPI mode 0, while the MLX90393 works on SPI mode 3.

Compared to alternative protocols like I^2C, magnetometers and IMUs (e.g., MLX90393 and ICM20948) usually support SPI with much higher clock (usually 7 MHz) than I^2C (usually 400 kHz at fast mode).

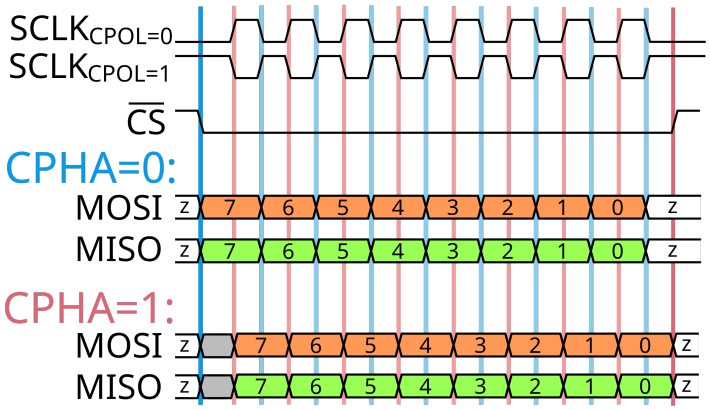


Figure 11: SPI timing diagram for both clock polarities and phases. Data bits output on blue lines if CPHA=0, or on red lines if CPHA=1, and sample on opposite-colored lines. Numbers identify data bits. Z indicates high impedance. (Source: [35])

Additionally, it is full-duplex compared to half-duplex of I^2C. Thus, using SPI can achieve much faster communication speed, which can effectively reduce the communication latency, enable more accurate timestamp logging within the testbed.

3.3.6 FPGA Communication Protocol: Universal Asynchronous Receiver/Transmitter (UART)

The FPGA must transmit sensor data to the host computer for further processing, and this is achieved using the Universal Asynchronous Receiver/Transmitter (UART) interface. UART is a full-duplex serial communication protocol that enables asynchronous data exchange between devices based on a predefined baud rate. A typical UART frame consists of a logic-low start bit, followed by 5–9 data bits, an optional parity bit, and one or two logic-high stop bits to indicate the end of transmission and reset the line (see **Figure 12**).

Due to its asynchronous nature and lack of robust error correction and addressing mechanisms, UART operates at relatively low speeds compared to more advanced interfaces. For example, with a baud rate of 115200 and standard framing, the effective throughput is approximately 11.52kB/s. Despite this limitation, UART is well-suited for the testbed's requirements. Transferring data from two magnetometers sampling at 25Hz only requires around 300bytes/s, which is well within UART's capacity. Its simplicity, low overhead, and minimal hardware resource usage make it an ideal choice for this application.

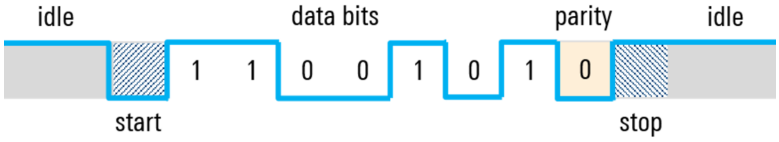


Figure 12: A typical UART frame, containing a start bit, data bits and an optional stop bit. This will be used to transfer sample data from the FPGA to a host computer. (Source: [36])

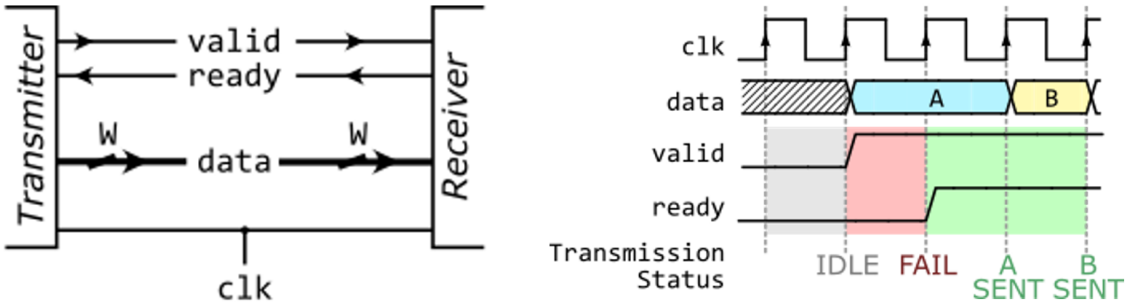
3.3.7 Other Essential Information for Testbed Implementation on FPGA

Ready / Valid Handshake For Data Transfer:

The Ready/Valid mechanism is commonly used for data flow control in FPGA-based data transfer systems, ensuring reliable communication between sender and receiver modules (See **Figure 13a**). This mechanism is widely used in the testbed design. It operates through two key signals:

- **Valid (Sender-controlled)**: Asserted high by the sender to indicate that the data on the bus is stable and valid.
- **Ready (Receiver-controlled)**: Asserted high by the receiver to signal its ability to accept new data.

Data transfer occurs only when both signals are simultaneously high at the rising clock edge. (See **Figure 13b**) The sender holds valid until the transaction completes, while the receiver throttles the flow by de-asserting ready (e.g., during buffer saturation).



(a) (b)

Figure 13: (a) The ready / valid handshake signals between a transmitter module and a receiver module. (b) Timing diagram showing the handshake process. Note that the data transfer will only begin when both valid and ready are high. This will be widely used for inter-module communication within the testbed design. (Source [37])

Dual Flip-Flop Synchroniser for External Sensor Interrupt Signal:

The Dual Flip-Flop (FF) Synchronizer is a widely used technique in FPGA designs to mitigate metastability risks during clock domain crossing for single-bit signals. In the testbed design, this is a potential issue on the external data ready (DRDY) interrupt signal from the magnetometer. When a signal transitions between asynchronous clock domains, the first FF captures the asynchronous input, potentially entering a metastable state if the setup time or hold time is violated (See **Figure 14**), while the second FF re-samples this output and significantly reducing the probability of metastability propagation. This introduces a deterministic latency of two destination clock cycles, ensuring stable signal transmission at the cost of minor delay.

Round-Robin Scheduling for Data Upload:

Round-robin scheduling is originally a method used for computer operation system process CPU resource scheduling. The scheduling algorithm allocates CPU time by cycling processes in a fixed order, granting each a set of time slice (e.g., 1 s) to execute, and when the time slice expires it is switched to the next task until all tasks finished (See **Figure 15**). An advantage of this method is the ensured equal resource allocated on each process.

This concept is borrowed for the testbed to equally allocate the UART transmission module to upload data from different IMU devices, and avoid sensor data being discarded due to overflow if it waits too long time before upload. In this application, the process will change into IMU device, and the time slices will become slices of a fixed number of bytes.

3.4 The Chopper-Pearson Method

The Clopper-Pearson method estimates the confidence interval for a binomial proportion p in binary outcome tests, where each trial yields strictly two results (e.g., ”pass” or ”fail”, ”success” or ”error”). For example, in experiments evaluating system reliability (e.g., counting successful operations out of n

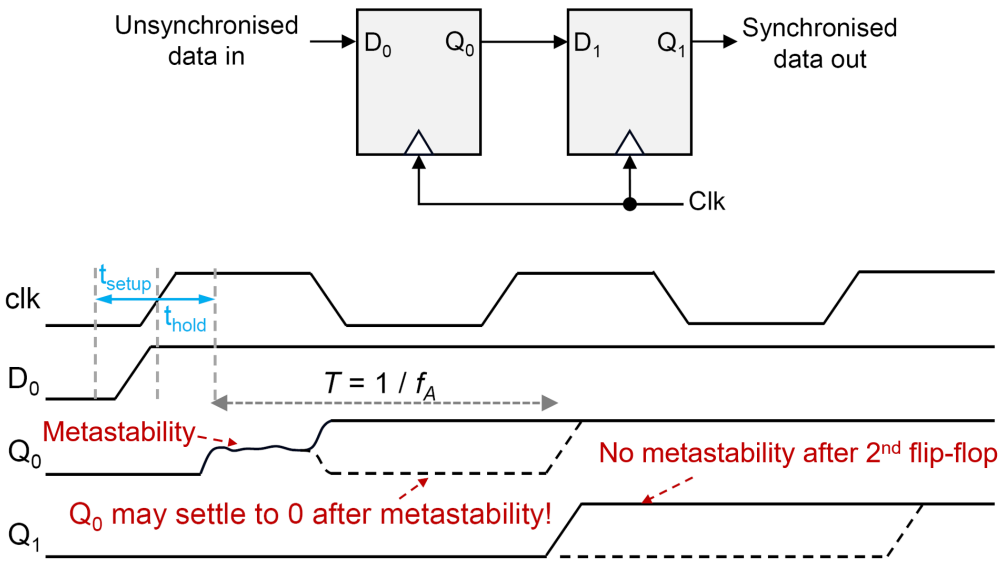


Figure 14: The structure of a typical dual-FF synchroniser, and the timing diagram showing of how it handles metastability when the input signal violates the hold or setup time. Note that this design is applied to every data ready interrupt signal from the sensor in the testbed design. (Source: [38])

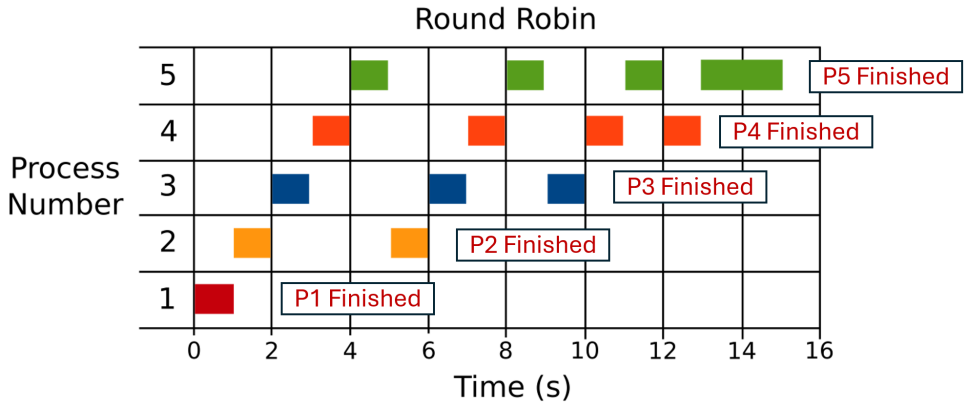


Figure 15: Round-robin scheduling in a computer operation system. Note that the CPU cyclically execute the processes (indicated by rectangular blocks), and every time only a small fixed time slice, until the process finishes. Similar concept is borrowed to schedule the UART transmitter in the testbed design. (Source: [39])

attempts) or medical diagnostics (e.g., positive/negative test results) [40, 41], it calculates the interval bounds as:

$$p_{\text{lower}} = B\left(\frac{\alpha}{2}; k, n-k+1\right), \quad p_{\text{upper}} = B\left(1-\frac{\alpha}{2}; k+1, n-k\right) \tag{2}$$

where $B(\cdot)$ is the beta distribution quantile function, k is the observed successes (e.g., passed tests), n is the total trials, and α defines the confidence level (e.g., 95% when $\alpha = 0.05$).

Binary tests with small sample numbers (e.g., fewer than 100 trials) are all applied with this method to enhance the accuracy of the result.

4 Methodology

4.1 Design of the Synchronisation Wand

The *Synchronisation Wand* is an embedded hardware solution to perform synchronisation for wearable IMU devices using the encoded EMP method (See **Figure 16**).

The wand is powered by an ESP32S3 microcontroller, and firmware programmed by ESP-IDF with FreeRTOS. With the on-board temperature compensated crystal oscillator (TCXO) and wifi synchronised timestamp via simple network time protocol (SNTP), the wand is able to provide an accurate and low-drift time reference for the IMU devices to align with and to get synchronised. The wand features an electromagnet module to send the EMPs for synchronisation, with high precision control signal generated by timer interrupt. It also has an on-board, 9-axis, ICM20948 inertial measurement unit (IMU) to support synchronisation using the conventional kinetic event method. The user interface is achieved by using four programmable push buttons and a 128x64 OLED screen. The file storage is enabled by an on-board 256 Mb flash storage and a micro-SD connector. For the power aspects, the synchronisation wand has a type 18650 li-ion rechargeable battery which is tested with duration of 40+ hours, and is rechargeable via the USB Type-C port. It also has deep sleep function that can minimise power consumption while still keeping system time when the device is unused. **Figure 17** shows the hardware architecture of the *Synchronisation Wand*, and **Figure 18** shows the PCB design.

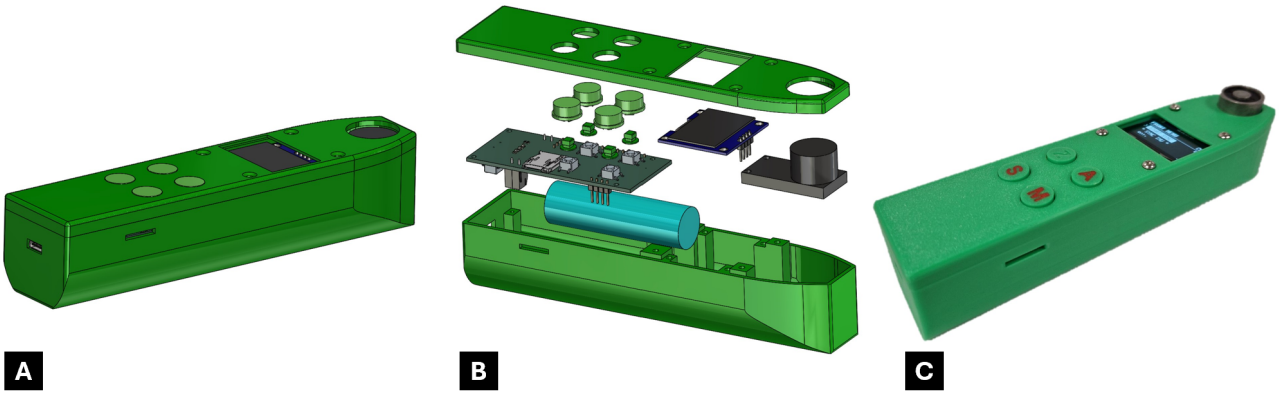


Figure 16: The synchronisation wand. (A) 3D CAD design of the wand, (B) exploded view, (C) A photo of the fully-assembled device.

Compared to the first version of the Synchronisation Wand, which's full design has been detailedly explained in the author's conference paper [13], the successive versions mainly added the Bluetooth low energy (BLE) support to the device and performed some firmware enhancements to the already-existing features. These are detailedly explained in the following subsections. All project files including the firmware source code are open-sourced and can be found on the GitHub page [42].

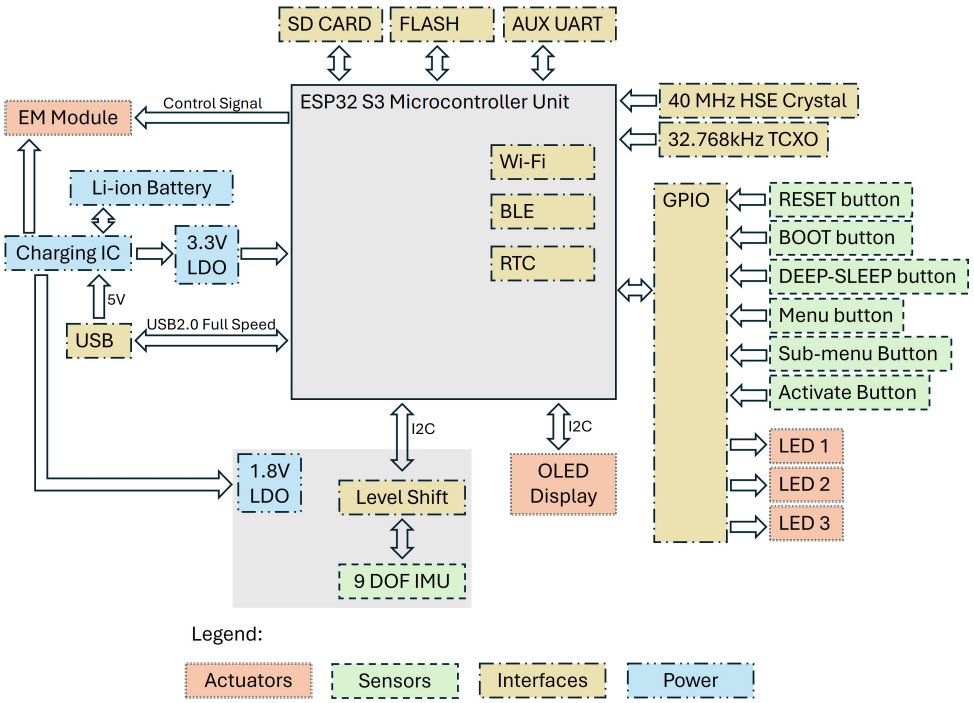


Figure 17: Hardware architecture of the wand. An ESP32-S3 controller is used, which internally contains the RTC for time keeping, Wi-Fi and BLE for communication. The power is provided by an Li-ion battery, and regulated using LDOs. User interface is achieved by using push buttons and OLED display. The EM module, powered by the battery, is controlled by a control signal from the ESP32-S3. The wand also contains an on-board ICM20948 IMU, which is operating in a 1.8 V domain.

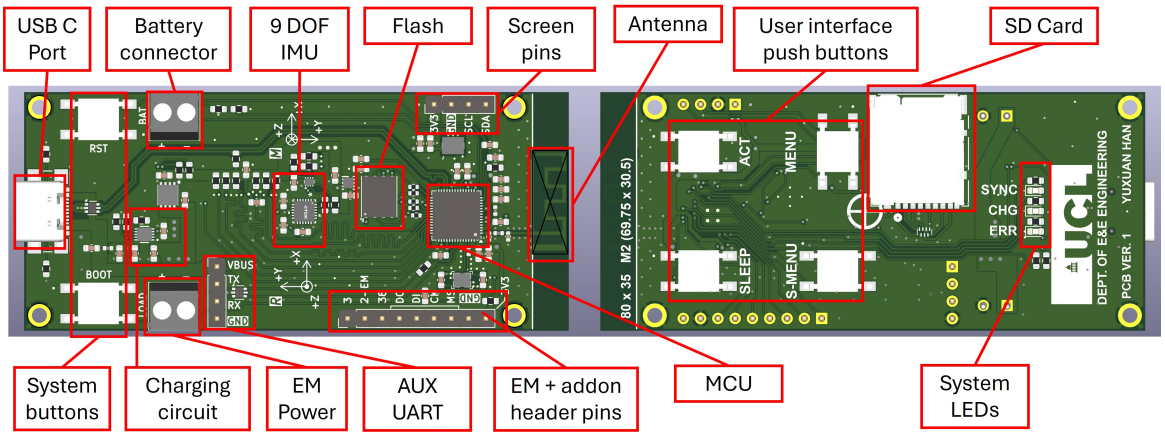


Figure 18: 3D rendered, populated printed circuit board with main components annotated. Note that the front side (left) is mainly populated with intergrated circuit chips, and the back side (right) is mainly populated with mechanical components like push buttons and the SD card connector.

4.1.1 Bluetooth Low Energy (BLE) Development

Bluetooth Low Energy (BLE) is developed to support real-time configuration of system parameters without reflashing the firmware and to improve the human-machine interface. The BLE is implemented on the top of the *Apache Mynewt NimBLE* host stack. The user can read and change the parameters in real time via Generic Attribute Profile (GATT) operations.

Generic Attribute Profile (GATT) Structure:

The tree structure of the GATT services and characteristics (See **Section 3.2.8** for explanations) is

shown in **Figure 19** below:

1. Device Information Service (*UUID*: 0x180A)
   └── **Manufacturer Name** (*UUID*: 0x2A29, Properties: Read)

2. Battery Service (*UUID*: 0x180F)
   └── **Battery Level** (*UUID*: 0x2A19, Properties: Read, Notify)
      └── ***Descriptor*: Client Characteristic Configuration** (*UUID*: 0x2902, Properties: Read, Write)

3. Wi-Fi Configuration Service (*UUID*: CAE6F78A-C2CF-4DEA-B1CF-4DB4208191A4)
   ├── **Wi-Fi SSID** (*UUID*: 645BE00E-CE39-4537-B6CC-050FE1978A02, Properties: Read, Write)
   └── **Wi-Fi Password** (*UUID*: C877D27C-1049-4314-8681-9AE1603F1693, Properties: Read, Write)

4. Synchronisation Parameters Service (*UUID*: CCD806BF-E77A-437A-99F6-D0293C791288)
   ├── **Point of Interest** (*UUID*: 2A5B8A41-D52F-472C-97FF-0C6F25C2B5C1, Properties: Read, Write)
   ├── **Maximum Synchronisation Error** (*UUID*: 4B43F4CA-A1A1-4B85-A4D1-78BA0F4248E9, Properties: Read, Write)
   ├── **Target IMU Sample Period** (*UUID*: 90636119-33BD-42DE-B02D-704A197ED724, Properties: Read, Write)
   ├── **Synchronisation Signal Pulse Width** (*UUID*: A92ADF1D-50A8-47CE-8F92-DF6489E66DE2, Properties: Read, Write)
   └── **Synchronisation Signal Duration** (*UUID*: FBFF0F62-E4B3-440F-8A9F-549BC0523C58, Properties: Read, Notify)
      └── ***Descriptor*: Client Characteristic Configuration** (*UUID*: 0x2902, Properties: Read, Write)

5. On-board IMU Settings Service (*UUID*: 61D91904-E6CE-4FFE-8000-8072F1F881CF)
   ├── **On-board IMU Status** (*UUID*: 0842E3D0-A040-4D66-80C8-95D2D3878D7D, Properties: Read)
   └── **On-board IMU Sample Period** (*UUID*: E22DC86F-F46C-40B6-9327-CFF9D3BA0857, Properties: Read, Write)

Figure 19: Generic Attribute Profile (GATT) tree showing key services and characteristics. The purple words shows the names of GATT services, the cyan words shows the UUIDs, the bolded black texts shows the names of GATT attributes, and at the end of each attribute, the properties are highlighted using green, red or yellow.

The user can monitor and adjust these parameters remotely via a BLE host device such as a mobile phone. The write operations of all characteristics should be in UTF-8 format. All numbers will be automatically converted into the correct formats. Float numbers are accepted. Note that it is the user's responsibility to ensure the input value is valid to avoid potential firmware malfunction. Every time a synchronisation-related parameter (e.g., *Maximum Synchronisation Error*) is updated, the synchronisation signal duration will be re-calculated automatically, and the host will be notified if the corresponding characteristic is subscribed to BLE notifications.

GATT Operations Handling:

GATT operations are handled using GATT characteristic callbacks. Every time there is a GATT operation, e.g., read or write, a callback function corresponding to the target characteristic will be triggered. **Figure 20** below shows an example flow chart of how the GATT operations of a synchronisation parameter-related characteristic are handled.

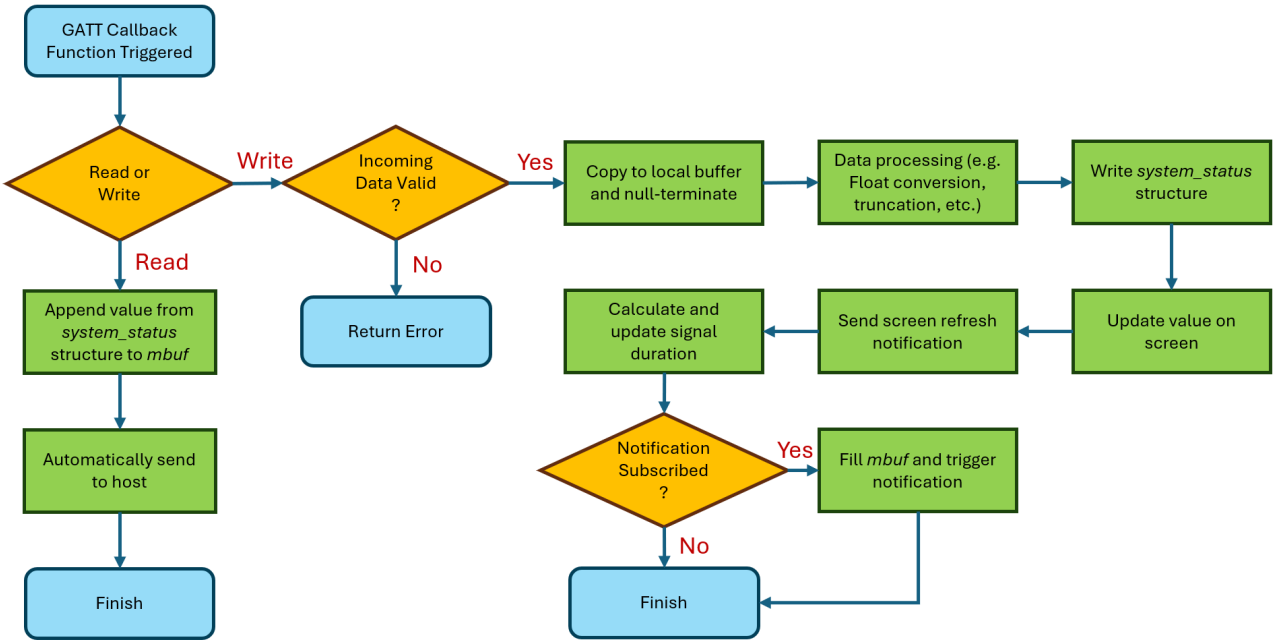


Figure 20: Example flow chart of how the GATT operations of a synchronisation parameter related characteristic is handled.

4.1.2 BLE Functionality Validation

To enable rapid verification of the Bluetooth Low Energy (BLE) module, a testing method based on mobile toolchain integration and serial log analysis is applied. The testing environment is configured as a standard 10-meter line-of-sight (LOS) scenario, utilizing the following equipment: a smartphone (Android/iOS) running nRF Connect v4.3, and the Synchronization Wand connected to a computer via USB serial monitor. The procedure comprises three phases:

- **Basic Connectivity Testing**: Perform 50 pairing attempts at both short-range (1 meter) and long-range (10 meters) scenarios. Calculate connection success rates (passing criterion: $\geq 95\%$ accuracy).
- **Signal Strength Verification**: Fix the Wand position, place the smartphone at 10 meters, and monitor real-time received signal strength indicator (RSSI) via nRF Connect over 2 minutes (passing criterion: average RSSI $\geq$ -80 dBm, several outliers accepted).
- **Parameter Configuration Stress Test**: Iteratively using BLE to configure the synchronisation parameters on the synchronisation wand for 100 times at 1 meter distance, and monitor whether the configuration is successful via the OLED screen. (passing criterion: $\geq 95\%$ success rate). If the firmware crashes during the session, the test will also be classified by fail.

The accuracy value based on the binary test results are all estimated by applying the Clopper-Pearson method with a confidence level of 95%. The verification results can be found in **Section 5.1**.

4.1.3 Other Firmware Enhancements

Additional firmware enhancements have been made to improve the system stability, synchronisation accuracy and user experience. This include the external low speed oscillator driver, the SD detect logic and the on-board IMU sample timing. Additionally, *Segger SystemView* support is configured to enable easier debugging and system resource monitoring. This can be enabled via UART at any general purpose input / output (GPIO) pin, or the joint test action group (JTAG) of either exposed pins or internal via the USB connection.

4.2 Design of the FPGA Based Testbed

After finishing implementing the synchronisation wand, this section will describe the design and implementation of the FPGA based testbed, which is a critical member within the systematic verification of the encoded EMP synchronisation method (See **Section 3.3.2**). This section will first present an overview of the testbed, describing its structure and internal timing at a higher level and then the implementation of its internal modules are explained. At the end of this section, some tests of the basic functionalities of the testbed will be setup for performance and timing evaluation. The full version of the verilog code used can be found on the GitHub repository for the synchronisation wand [42].

4.2.1 System Overview

The main purpose of the FPGA based testbed is to emulate two IMU devices, operating with predefined timing error, as a standard value to compare with the EMP synchronisation results to verify its accuracy. The system overview of the testbed design is shown in **Figure 21** below:

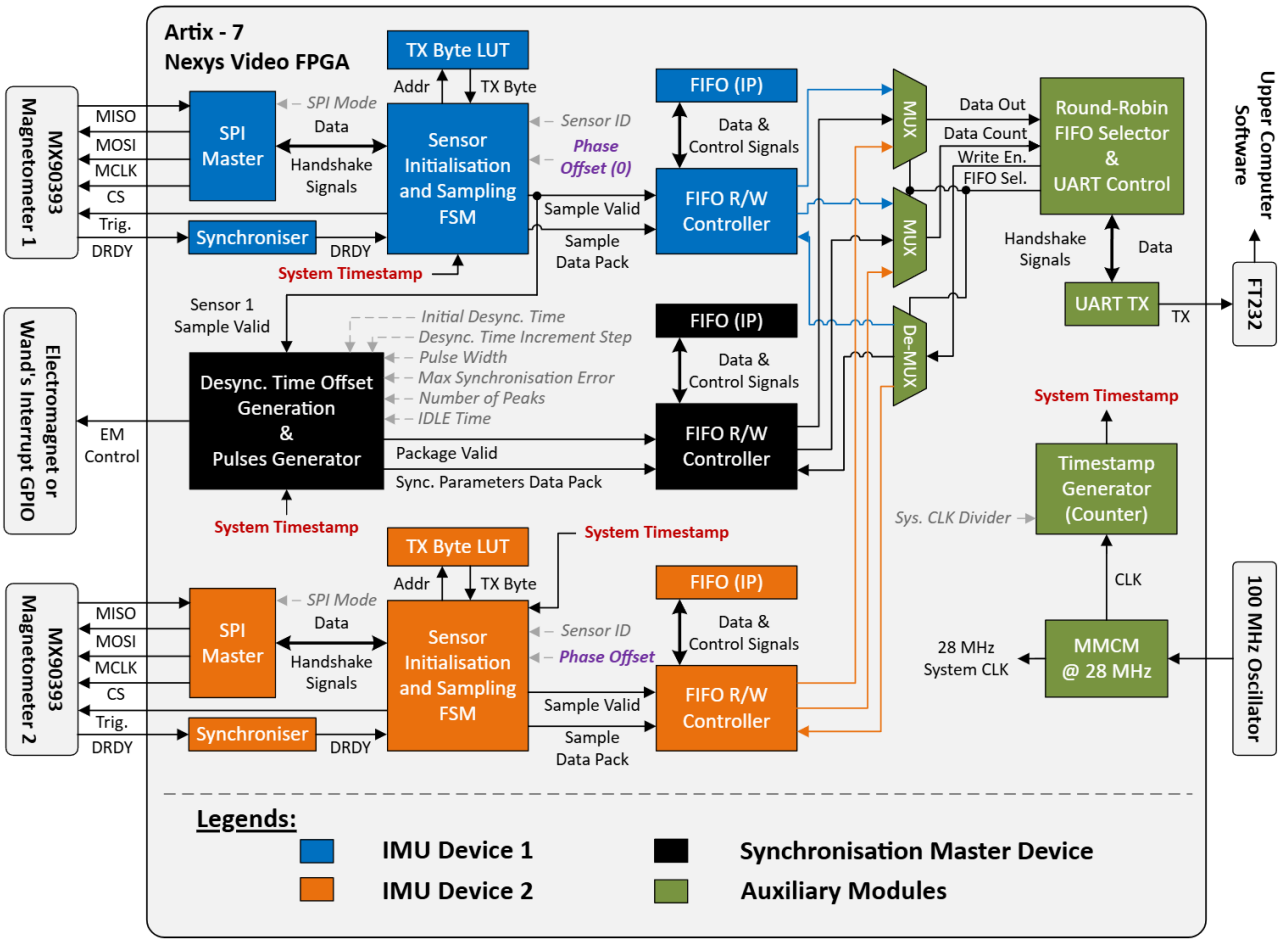


Figure 21: Overview of the FPGA based testbed for encoded EMP synchronisation verification, with only important signals displayed. Note that the internal modules are divided into four sections, highlighted in four different colours and explained in the legends. These include two emulated IMU devices, one synchronisation master device, and auxiliary modules.

This testbed implementation can be divided into four different parts internally, highlighted with different colours on the figure:

- **Two emulated IMU devices (IMU device 1 & 2)**: Identical designs, with basic sampling trigger control, sample data acquisition and timestamping functionalities. They operates with

configurable pre-defined timing error offset ($T_{\mathbf{PHASE}}$). Note that although they are called "IMU devices", sometimes they are referred as "Magnetometers" in the report since only magnetometers are used.

- **A synchronisation master device**: The master device generates the electromagnet control signal for encoded EMP synchronisation (See **Section 3.1.2**), which also operates with a configurable pre-defined timing error with IMU device 1 ($T_{\mathbf{OFFSET}}$).
- **Auxiliary Modules**: The auxiliary modules are mainly responsible for transmitting data onto the upper computer, and FPGA internal timestamp generation.

The magnetometers operate in single measurement mode, with their sampling precisely controlled by the testbed via dedicated trigger signals. Within this system, IMU device 1, IMU device 2, and the master device all operate with intentional timing error offsets relative to one another. This relationship is illustrated in the timing diagram shown in **Figure 22**.

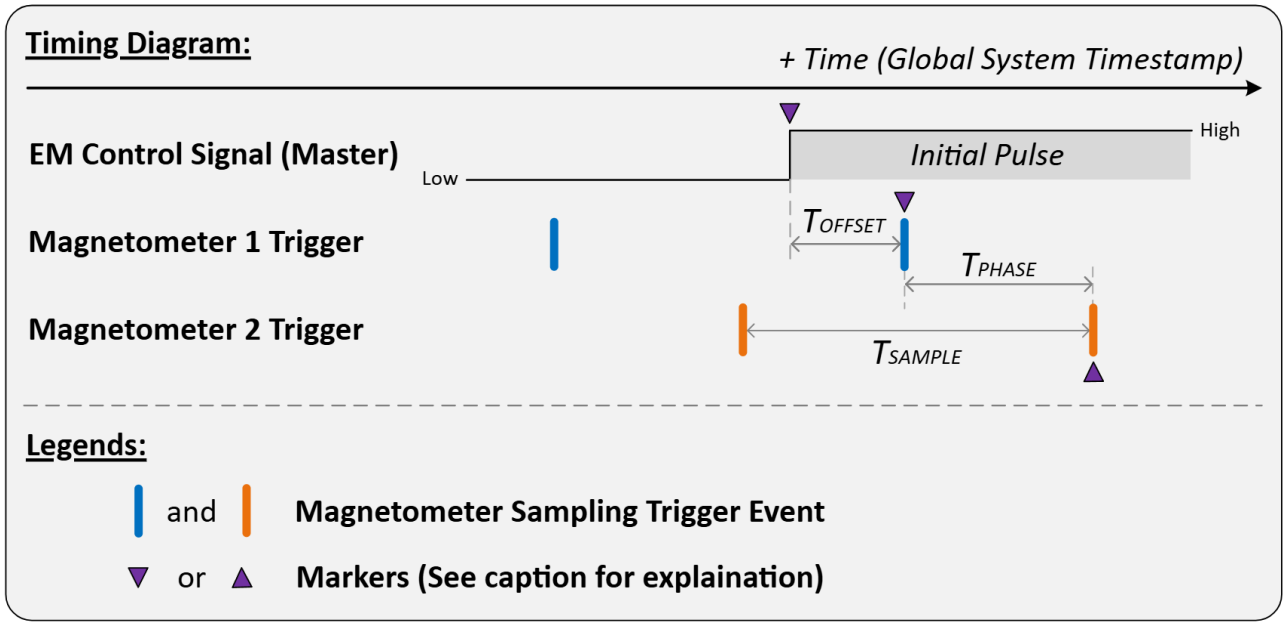


Figure 22: This timing diagram illustrates the sampling trigger events of two IMU magnetometers and the operation of the master device (EM Control Signal) within the testbed, all referenced against the global system timestamp generated by auxiliary modules. The purple triangular markers represent events that each device perceives to be simultaneous. However, due to local clock desynchronisation, these events do not align from a global perspective.

As shown in the figure, the timing error offset between the master device and magnetometer 1 is defined as T_{OFFSET}. Similarly, the timing error offset between magnetometer 1 and magnetometer 2 is defined as T_{PHASE}. The proper definitions of these two timings are shown in **Table 2**. Both T_{OFFSET} and T_{PHASE} are precisely configurable within the testbed using hardware timers.

Timing Name	Definition
$T_{\mathbf{OFFSET}}$	The time interval from the first rising edge of the electromagnet control signal to the **NEXT** trigger event for magnetometer 1.
$T_{\mathbf{PHASE}}$	The interval from the trigger event of magnetometer 1 to the **NEXT** trigger event of magnetometer 2.

Table 2: Timing Definitions for T_{OFFSET} and T_{PHASE}.

4.2.2 The Timestamp Generator

The timestamp generator generate 40-bit real-time global system timestamps with default resolution 1 μs from a divided system clock. Its input / output properties are shown in **Figure 23**, and explained in **Table 23** below:

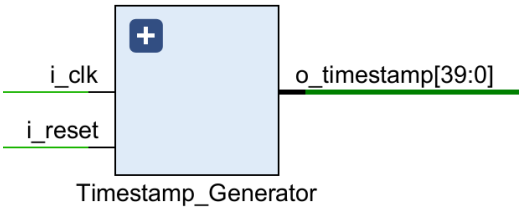


Figure 23: Input / output properties of the timestamp generator module.

Wire Name	Type	Explanation
i_clk	input	System main clock input.
i_reset	input	System reset signal input.
o_timestamp	Output	40 bit system timestamp, with default resolution of 1 μs

Table 3: The table explaining the input and output wires of the timestamp generator

This Verilog module implements clock division and timestamp generation using a dual-counter structure. A first-stage counter divides the input clock by a configurable prescaler value to a lower frequency, then it is used to drive the second stage counter containing a 40 bit register, which's value increments by 1 every time when being triggered, generating the system timestamp and being output on `o_timestamp`. The core verilog code implementation of this part can be found in **Appendix A.1**.

4.2.3 The SPI Master

The SPI master is responsible for handling the transmission and reception of data with the magnetometer via SPI. Its input / output properties are shown in **Figure 24** and **Table 4** below:

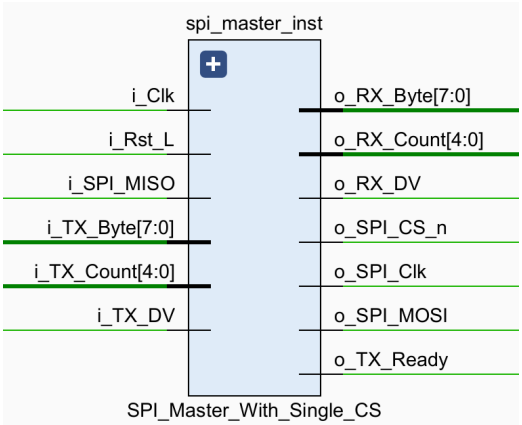


Figure 24: Input / output properties of the SPI master module.

Signal Name	Type	Description
i_Rst_L	Input	Active-low reset
i_Clk	Input	Main system clock
i_TX_Count	Input	Number of bytes to transfer per CS activation
i_TX_Byte	Input	8-bit data byte to transmit
i_TX_DV	Input	Data Valid to initiate transmission
o_TX_Ready	Output	Ready status flag
o_RX_Count	Output	Received byte counter
o_RX_DV	Output	Receive Data Valid pulse
o_RX_Byte	Output	8-bit received data byte
o_SPI_Clk	Output	SPI clock output
i_SPI_MISO	Input	Master-In-Slave-Out serial data line
o_SPI_MOSI	Output	Master-Out-Slave-In serial data line
o_SPI_CS_n	Output	Chip Select signal

1 Signal widths adapt automatically based on MAX_BYTES_PER_CS parameter
2 CLKS_PER_HALF_BIT must be 2 for proper timing closure
3 CS_INACTIVE_CLKS enforces minimum idle time between transactions

Table 4: SPI Master Interface Signals Description

External modules can communicate with the SPI master through ready / valid handshake (`i_TX_DV` and `o_TX_Ready`) to enable the data presented on the `i_TX_Byte` line to be transmitted through SPI. (See **Section 3.3.7** for the description of ready / valid handshake) For the SPI data reception part, the handshake is further simplified by assuming the destination modules are always ready, thus only an `o_RX_DV` is used which pulses high when a complete reception of 8 bits finished and can be read on `o_RX_Byte` .

Whenever the data valid handshake signal is received when the module is ready, it will start to perform the SPI transmission according to the SPI timing diagram shown in **Figure 11**.

SPI Clock Generation:

The SPI clock generation is achieved by using a counter, triggered by a divided clock from the system main clock. The SPI clock will start at either level low or high, defined by the SPI mode used (See **Section 3.3.5** for SPI mode, CPOL and CPHA definitions). The counter reversely counts from 16 to 0, and every time when it counts, it toggles the polarity of the SPI clock and finish the clock generation. (See **Appendix A.2** for more detail).

Transmission and Receiption:

The transmission and reception of the bits are achieved by using counter as well. The counter counts reverses counts from 8 to 0, recording the number of bits that still need to be transmitted / received. At appropriate SPI clock edges, depending on the SPI mode, one bit is shifted out / sampled (See **Table 1**). This is achieved by checking whether the following logics are true:

For transmission:

```
1    (r_Leading_Edge & w_CPHA) | (r_Trailing_Edge & ~w_CPHA)
```

For reception:

```
1    (r_Leading_Edge & ~w_CPHA) | (r_Trailing_Edge & w_CPHA)
```

where on leading edge `r_Leading_Edge` is high `r_Trailing_Edge` is low, vice versa. More details on the verilog code are shown in **Appendix A.2.2**.

Chip Select:

The chip select signal (`o_SPI_CS_n`) is controlled by a state machine that follows the flow chart in **Figure 25**:

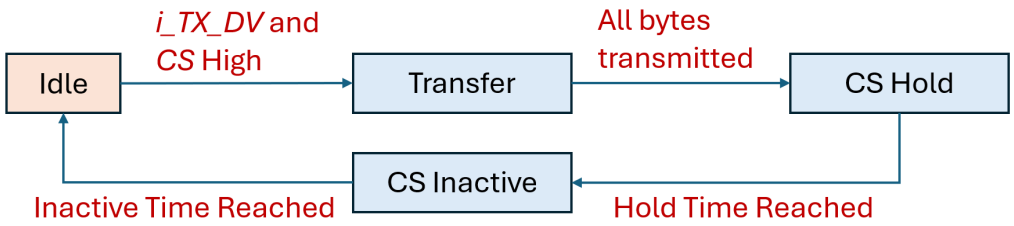


Figure 25: A flow chart summarising the operation of the chip select finite state machine. Note that the state machine will only progress to the next state if the conditions specified by the red words are satisfied. The default state is indicated by the orange block.

This state machine comprises four states: Idle, Transfer, CS Hold, and CS Inactive. When both the chip select signal and the transmission data valid signal (`i_TX_DV`) are asserted high, the state machine transitions from Idle to the Transfer state, simultaneously de-asserting CS. Once the specified number of bytes defined by `i_TX_Count` have been transmitted, the state machine enters the CS Hold state, where it waits for a predefined duration to satisfy the chip select hold time required by the slave chip. It then transitions to the CS Inactive state, reasserting CS high and remaining in this state long enough to meet the minimum chip select inactive duration required by the slave chip. After this delay, the state machine returns to the Idle state, ready to initiate a new transmission cycle.

4.2.4 The Emulated IMU Device

The emulated IMU device (See **Figure 21** for IMU device 1 and IMU device 2) can be divided into three sections according to their functionalities:

1. **Sensor Communication**: Made up with the SPI master (See **Section 4.2.3**) and a dual-FF synchroniser to handle the communication with the external magnetometer chip.
2. **Sample Acquisition**: Made up with a sample acquisition finite state machine and a look up table. Its key roles include sensor initialisation, timing error offset generation, sampling control and sample data packet generation.
3. **Data Buffering**: Made up with a first-in, first-out (FIFO) buffer and its controller, this section is responsible for splitting the sample data packets from the sample acquisition module into individual bytes. These bytes are temporarily stored in the FIFO buffer, awaiting transmission to the host via UART.

Among these, only the most important sample acquisition module will be described in detail in the following sections.

Look Up Table (LUT) for Byte Transmission:

The LUT operates by inputing an address and outputting the stored value. It is used here to store the bytes that needed to be transmitted to the magnetometer to perform initialisation and sampling. (See **Appendix A.3** for verilog code).

Delay Mechanism Implementation:

To achieve precise time delays between magnetometer triggers, a dedicated hardware timer module is implemented. At its core, the timer consists of a counter driven by the system's main clock. The

timing duration is defined by the parameter `timer_autoreload_value`, which specifies the number of clock cycles to count before timing completes. The timer operation is similar to the one represented on **Figure 3** with the only difference that it will only run for one "Time Interval" on the figure.

The timer is controlled via the flag `timer_IR_flg`. When this flag is set low, the timer begins counting. Once the counter reaches the `timer_autoreload_value`, `timer_IR_flg` is automatically set high to indicate that the delay period has elapsed.

In addition, the timer includes a compare-match output, `timer_CC1_flg`, which generates a single-cycle pulse when the counter value matches a user-defined `timer_CC1_value`. This can be used to trigger intermediate events with high timing precision. For verilog implementation, see **Appendix A.4**.

Setting the timer to perform delay can be achieved by using a dedicated states inside a finite state machine. This is shown in **Figure 26** below, highlighted with orange colour. When transitioning into the delay state, the delay time is configured and at the same time the timer is triggered by setting the flag to zero. The state machine will being "trapped" by the delay state until the flag turns high, indicating the delay time reached.

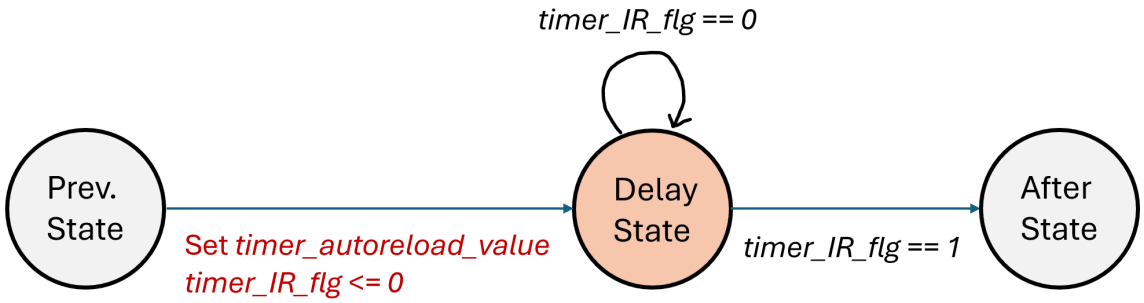


Figure 26: A state machine showing how to use a dedicated delay state to "trap" the finite state machine and achieve delay using the timer. Note that the actions during state transition are annotated in red text, and the conditions are annotated in black text.

Sensor Initialisation and Sampling Finite State Machine:

The behaviour of the sample acquisition finite state machine is summarised in the flowchart in **Figure 27**. Note that the real verilog implementation of this module includes more auxiliary states and transitions.

When the finite state machine (FSM) starts, it first sends an instruction via SPI to terminate any ongoing measurement. It then enters a delay stage, during which **the IMU device 2 is held for an additional $T_{\mathbf{OFFSET}}$ compared to the IMU device 1**. This effectively injects a timing error offset of T_{OFFSET} between the sampling times of the two magnetometers.

After completing sensor reset and configuration, the FSM transitions to the sampling stage. Each sampling cycle is governed by a timer configured to T_{SAMPLE}. Within each cycle, the FSM first **asserts the sampling trigger and simultaneously records a timestamp**. It then waits for the measurement to complete, retrieves the data, constructs a data packet, and transmits it to the FIFO controller. The FSM then waits for the timer to elapse before initiating the next sampling cycle.

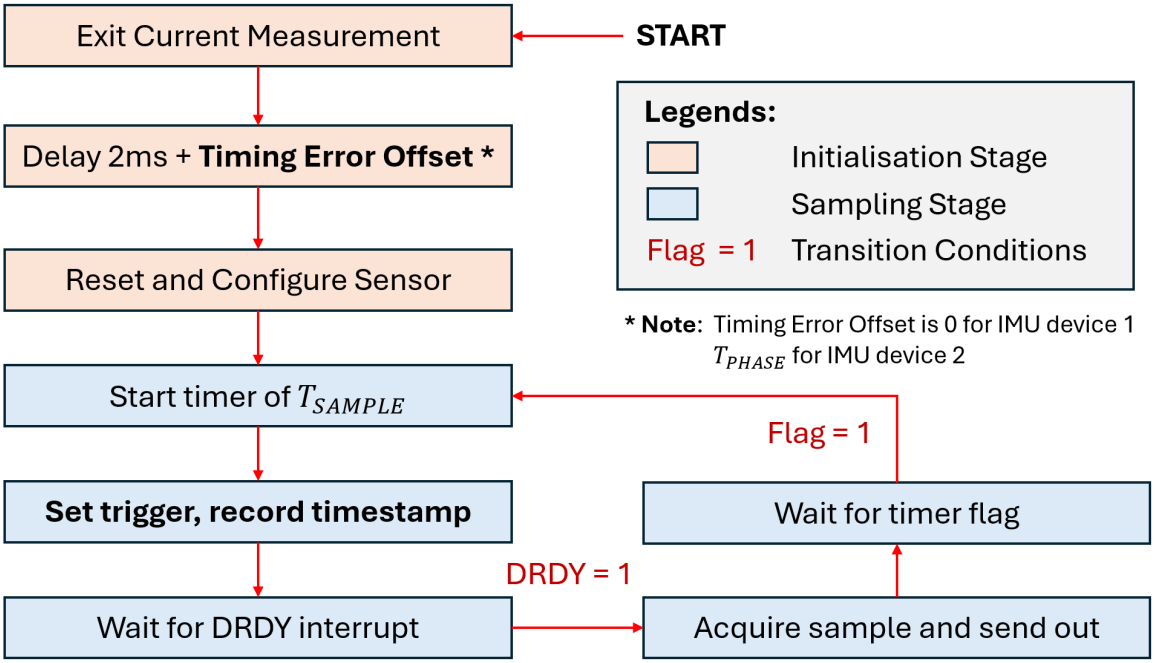


Figure 27: The flow chart summarising the operation of the sensor initialisation and sampling finite state machine. Note that during initialisation stage, different timing error offsets are added to two different IMU devices to create T_{PHASE}. During sampling stage, timestamping is done when setting the trigger.

Sample Data Packets:

The acquired sample and its corresponding timestamp are packed into a 192-bit data packet within the sensor initialisation and sampling finite state machine (FSM), following the structure shown in **Figure 28**. This packet is then sent in parallel to the FIFO controller module, which segments it into individual bytes and stores them in a FIFO buffer, ready for transmission via UART.

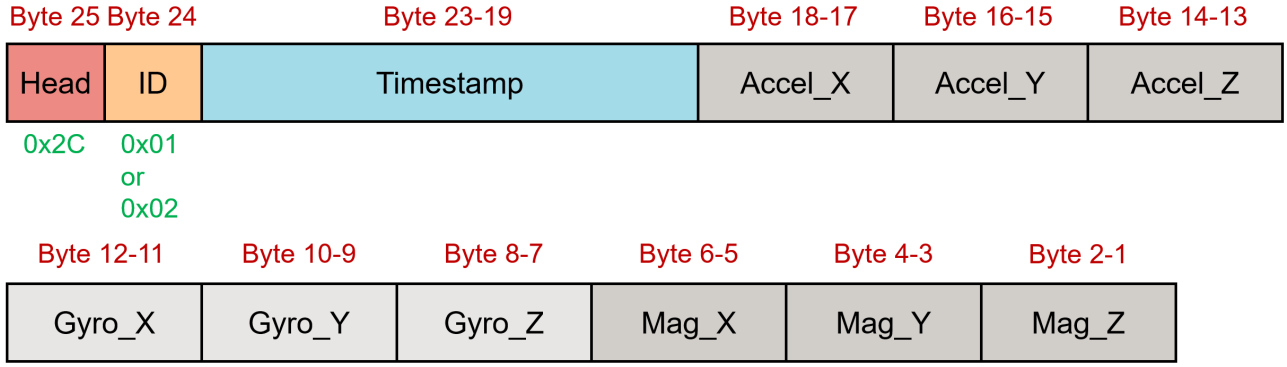


Figure 28: The structure of a 192-bit data packet from the emulated IMU device. A typical packet contains one head byte, an identifier byte, a 5-byte timestamp and sample data.

Each packet begins with a header byte `0x2C`, which serves as a marker to help the host software reliably identify the start of a new packet during decoding. The header is followed by a 1-byte device identifier: `0x01` for IMU device 1 or `0x02` for IMU device 2, allowing the host to determine the source of the data. The remaining bytes consist of a 5-byte timestamp and the associated sample data.

4.2.5 The Synchronisation Master Device

The synchronisation master device is primarily responsible for generating the encoded EMP synchronisation signal. To support automated testing, it is designed to repeatedly perform synchronisation

test cycles without manual intervention. One of its key features is the ability to introduce a configurable timing error offset T_{OFFSET} between itself and IMU device 1. This offset can be automatically adjusted between tests in fixed increments, as specified by the user.

In addition, the master device generates data packets containing synchronisation event metadata, including the current T_{OFFSET} value and a timestamp marking the end of each experiment session. These packets allow the host system to efficiently segment and analyse individual tests from continuous data logs.

The Main Finite State Machine:

The behaviour of the synchronisation master device finite state machine is summarised in the flowchart in **Figure 29**. Note that the real verilog implementation of this module includes more auxiliary states and transitions.

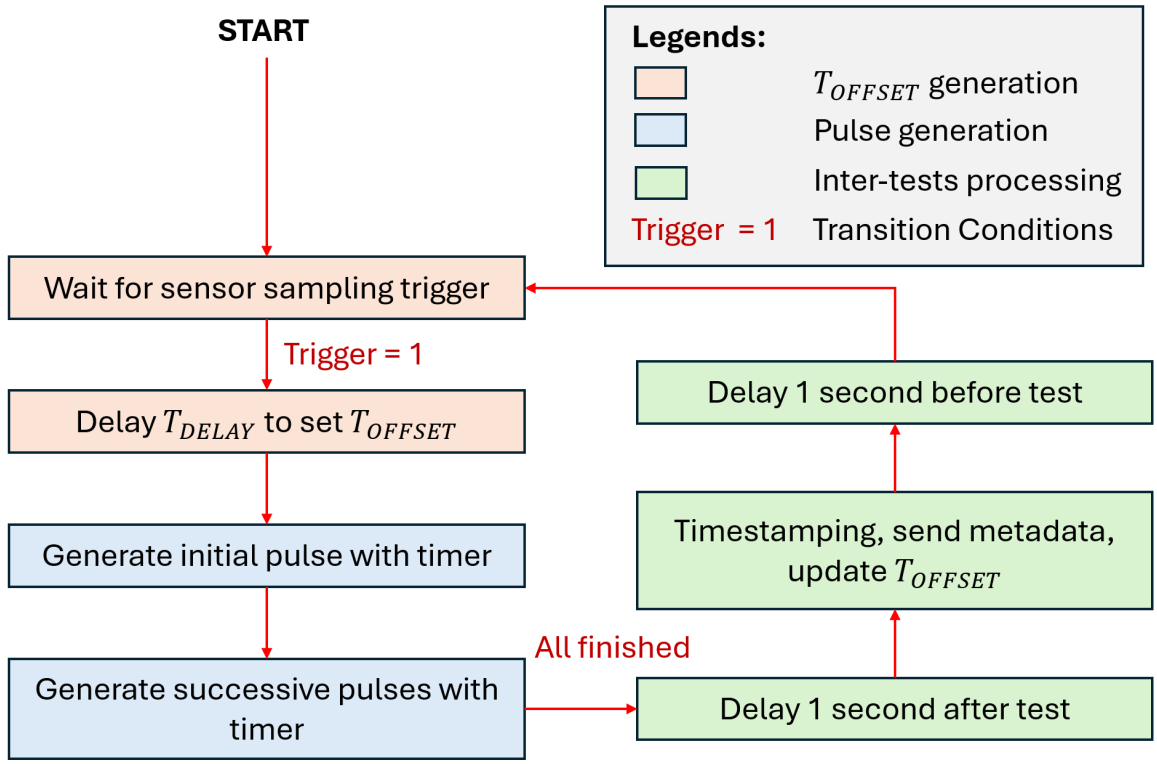


Figure 29: The flow chart summarising the operation of the finite state machine inside the synchronisation master device. Note that the orange blocks shows the T_{OFFSET} generation stage, the blue blocks shows the synchronisation pulse generation stage, and the green blocks shows the inter-tests processing stage.

The process of generating T_{OFFSET} can be better understood with the aid of the timing diagram shown in **Figure 30**. As illustrated, the defination of T_{DELAY} is given in **Table 5**:

Timing Name	Definition
T_{DELAY}	The time interval between the first rising edge of the encoded EMP synchronisation pulse sequence and the **preceding** sampling trigger event of magnetometer 1

Table 5: Timing Definitions for T_{DELAY}.

Given the known sampling period T_{SAMPLE} and the desired offset T_{OFFSET}, T_{DELAY} is calculated as:

$$T_{\text{DELAY}} = T_{\text{SAMPLE}} - T_{\text{OFFSET}} \tag{3}$$

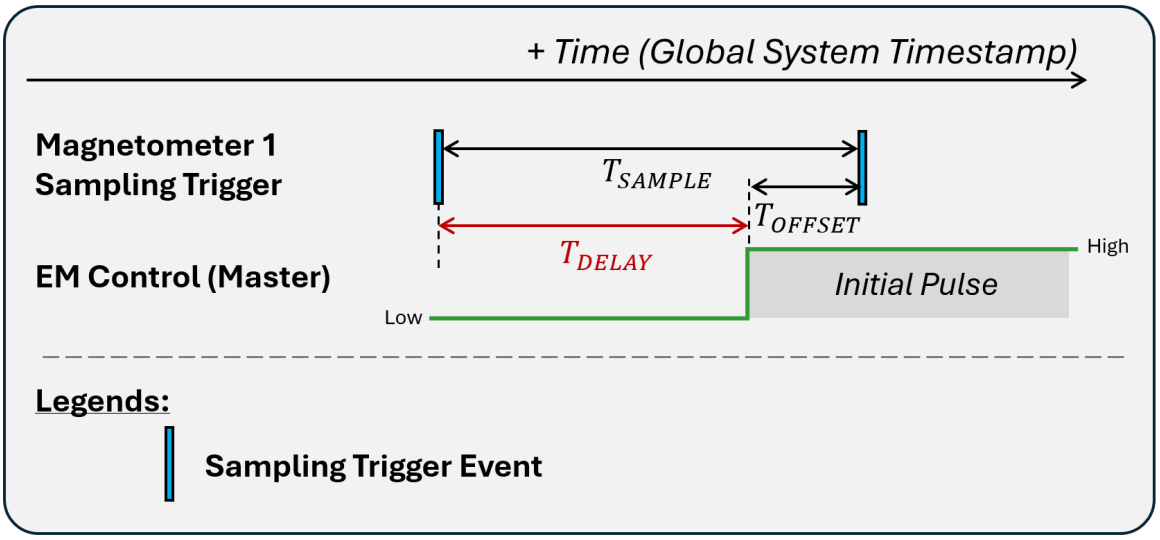


Figure 30: The timing diagram demonstrating how T_{OFFSET} can be set by using a delay with duration T_{DELAY} after a sampling trigger signal from magnetometer 1 (the blue block) is detected.

To implement this offset, the finite state machine first waits for a sampling trigger from magnetometer 1, then delays by the calculated T_{DELAY} before initiating the synchronisation pulse sequence.

The synchronisation pulses are generated using delay timers. Each time the timer reaches the programmed interval corresponding to the pulse width, the polarity of the synchronisation signal line is toggled, thereby generating the desired pulse sequence.

Once all pulses are completed, a post-test delay of 1 second is applied. The system then records a timestamp and packed it together with the current T_{OFFSET} into a metadata packet, which is sent to the FIFO controller and queued for transmission to the host. Afterward, T_{OFFSET} is incremented by a user-defined step size. A pre-test delay of 1 second is then applied before the next test cycle begins.

To prevent overflow (i.e., when $T_{\text{OFFSET}} > T_{\text{SAMPLE}}$), the offset value is automatically reset to its initial value, as defined by the user.

Metadata Packets:

The metadata packet structure for the synchronisation master device is shown in **Figure 31**. Similar to the packet for emulated IMU devices, it is also 24 bytes long. The difference are the identifier changed to `0xFF`, and some bytes originally used for sample data are used to store T_{OFFSET}.

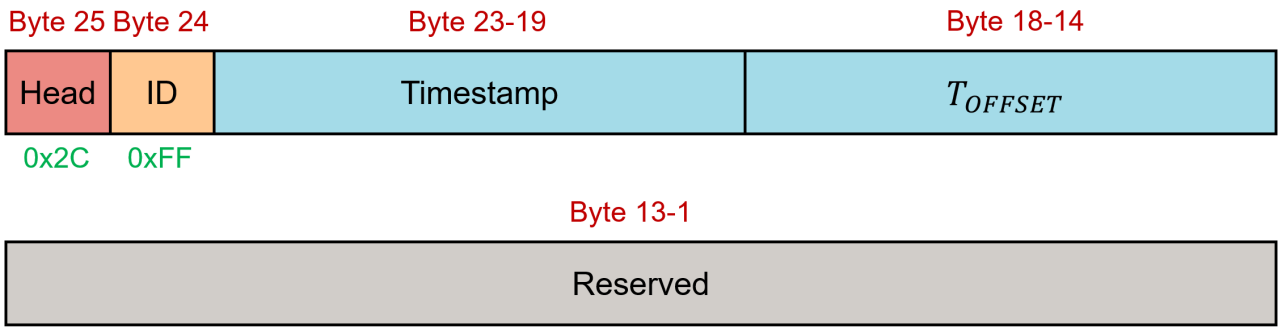


Figure 31: The structure of a 192-bit data packet from the synchronisation master device. A typical packet contains one head byte, an identifier byte, a 5-byte timestamp, a 5-byte T_{OFFSET} and reserved bytes.

4.2.6 The UART Transmitter

The UART transmitter is responsible for transmitting the data packets of the emulated IMU devices and the synchronisation master device to the computer host. The UART transmitter module is shown in **Figure 32**, and its input / output pin properties are explained in **Table 6**.

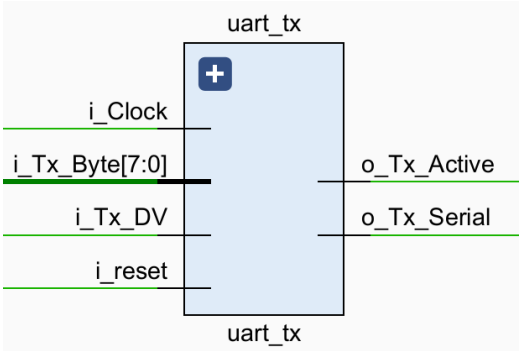


Figure 32: Input / output properties of the UART transmitter module.

Signal Name	Type	Description
i_Clock	Input	Main system clock
i_Tx_DV	Input	Transmit Data Valid
i_Tx_Byte[7:0]	Input	Byte to be transmitted
i_reset	Input	Active-low reset
o_Tx_Active	Output	Transmission Status Indicator (Ready signal)
o_Tx_Serial	Output	TX serial output of the UART
o_Tx_Done	Output	Transmission complete pulse flag

Table 6: UART Transmitter Interface Signals Description

Ready / valid handshake (See **Section 3.3.7**) is used to interact with the module and trigger transmission. The baud rate is configured and implemented using counters, and the finite state machine used for generate a UART signal according to the specified timing diagram (See **Figure 12**) is shown in **Figure 33**.

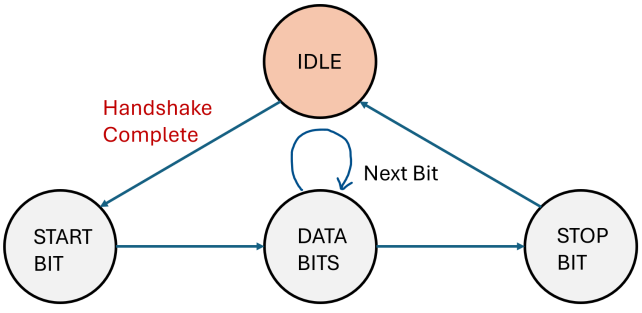


Figure 33: The finite state machine that used for the implementation of the UART transmitter module. The default state is marked with orange, and the state transition condition is annotated using red text.

4.2.7 The Round-Robin FIFO Selector and UART Controller

The round-robin FIFO selector and UART controller is a module that allocates UART transmission access among multiple FIFO buffers, enabling data packets to be transmitted from different sources in a fair and orderly manner. Its integration within the system is illustrated in **Figure 21** (notice the green block on top-right, plus the two multiplexers and one de-multiplexer), and the scheduling

behaviour is described in **Section 3.3.7**.

To implement the round-robin scheduling logic, the module uses a finite state machine (FSM), shown in **Figure 34**. The FSM cyclically checks each FIFO buffer for data availability, this is being done by checking whether the 8-bit wide FIFO buffer contains equal or more than 24 data (i.e. a complete 144-bits data packet present). Once a complete data packet is found, the module initiates transmission of the complete 24-byte packet via the UART transmitter to the host device.

To ensure fair sharing of the UART interface among the three data sources, the module enforces strict round-robin scheduling: after a packet from one FIFO buffer is transmitted, the module skips any subsequent packets in the same buffer during that cycle. Instead, it proceeds to the next FIFO buffer in sequence, maintaining balanced access and avoiding transmission starvation.

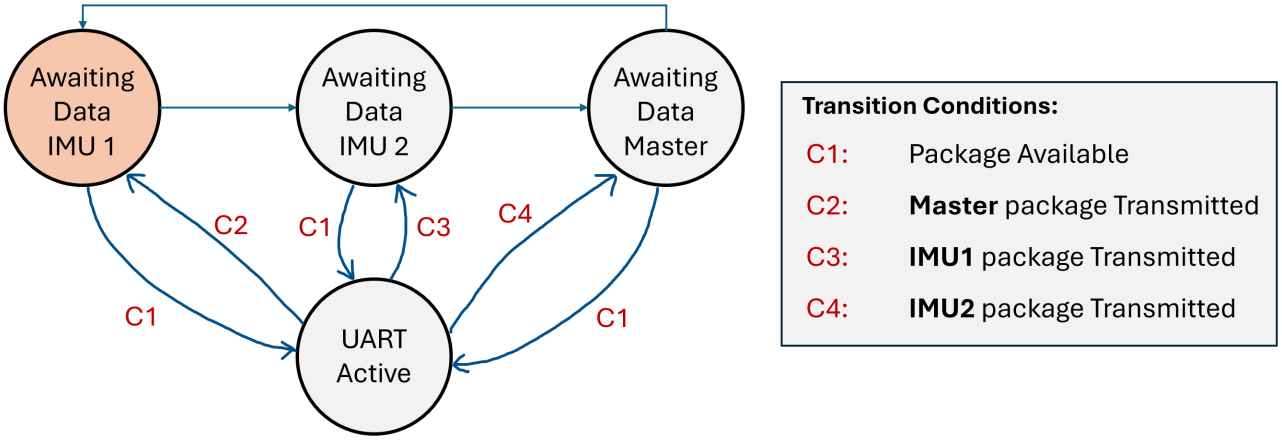


Figure 34: Finite state machine that used within the round-robin FIFO selector and UART controller. Note that the initial state is highlighted in orange, and the transition conditions are marked in red. The state machine will keep check the packet availability of each FIFO buffer circularly, and perform transmission when available.

4.2.8 Testbed Functionalities Verification

Before employing the testbed to validate and analyse the encoded EMP synchronisation method, its core functionalities must first be verified. The verification consists of three components: evaluating the accuracy of the synchronisation pulses, assessing the timing precision of the magnetometer sampling triggers, and stress test to observe the stability of the testbed when running automated testing over an extended period (i.e., 400 individual tests for 2+ hours).

The testbed is configured with the following parameters: $T_{\text{SAMPLE}} = 40\,\text{ms}$, $T_{\text{PHASE}} = 2.75\,\text{ms}$, $w = 320\,\text{ms}$, and $a = 10\,\text{ms}$. For the automated test running, initial T_{OFFSET} is configured to start from 0 ms, incrementing with a step of 0.1 ms. The automated testing and sample acquisition on the testbed will be left running for 2+ hours containing 400+ individual tests. The test results can be found in **Section 5.2**.

Synchronisation pulse accuracy:

To verify the accuracy of the synchronisation pulse, an oscilloscope (Rhode & Schwarz RTB2004) is used to probe the waveform and measure key timings via time cursors. The durations of the initial and first pulses are then compared against the expected theoretical values to confirm their accuracy.

Sampling Trigger Accuracy:

The sampling trigger timing is verified through timestamp analysis. To evaluate the sampling period precision of each magnetometer, the time differences between successive samples from either IMU device 1 or IMU device 2 are computed. The resulting distribution of T_{SAMPLE} should be a sharp

peak at 40 ms. To assess the timing offset between devices, the differences between adjacent samples from IMU device 1 and IMU device 2 are calculated. This yields the distribution of T_{PHASE}, which is expected to be a sharp peak as well at 2.75 ms.

Automated Testing Function Reliability:

The reliability of the automated testing function can be evaluated by analysing the trend of T_{OFFSET} values recorded in the received metadata packets. In the absence of data loss, a perfect linear progression should be observed from the initial 0 ms setting up to 40 ms, followed by an automatic reset of the value back to 0 ms. This pattern indicates that the automated testing system operates stably without crashing and correctly adjusts the test parameters between runs.

4.3 Sensor Configuration and Timing Analysis

With the timing behaviour on the FPGA side properly configured in the previous section, this section focuses on the timing characteristics within the magnetometer chip that may impact the overall precision of the verification platform design. It begins by introducing the issue of measurement window time lag inherent to the magnetometer. Then, the sensor configuration is presented and the time lag is quantified. Finally, a dedicated experiment is proposed to assist analysing how variations in this time lag affect the timing resolution of the verification platform.

4.3.1 Time Lag Between Sampling Trigger and Magnetometer Measurement Window

While the testbed supports configurable timing offsets between the master and the two emulated IMU devices (T_{OFFSET} and T_{PHASE}), an additional timing factor must be considered to obtain the true desynchronisation between devices: the inherent time lag between the sampling trigger and the actual start of the magnetometer's measurement window (see **Figure 9**). This delay, denoted as T_{LAG}, is illustrated in the timing diagram in **Figure 35**. As a result, the effective timing offset between IMU device 1 and the master becomes:

$$T_{\text{DESYNC}} = T_{\text{OFFSET}} + T_{\text{LAG}} \tag{4}$$

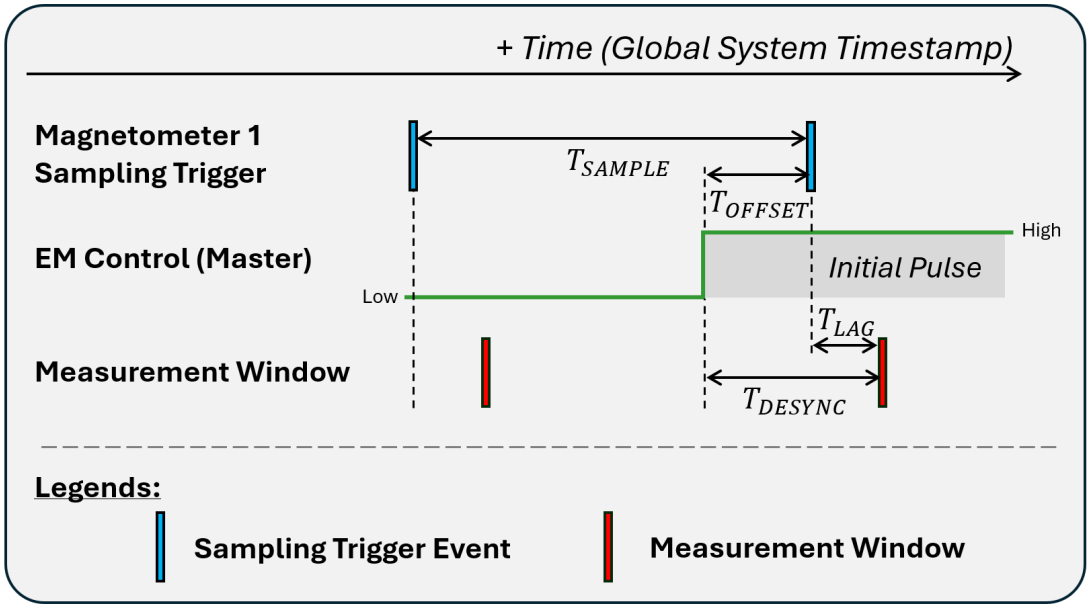


Figure 35: This timing diagram illustrates the timing relationship between the trigger signal event, EM control signal and the real measurement window. Note that the real sample measurement acturally happen with a delay of T_{LAG} after the sampling trigger event. This need to be considered and correctly conpensated in the verification system design.

The definitions of these two timing quantities are summarised in **Table 7**:

Timing Name	Definition
T_{LAG}	The time interval between the magnetometer sampling trigger assertion and the real measurement window.
T_{DESYNC}	The time interval from the first rising edge of the encoded EMP synchronisation pulse sequence to the real sample measurement window. $T_{\text{DESYNC}} = T_{\text{OFFSET}} + T_{\text{LAG}}$

Table 7: Timing Definitions for T_{LAG} and T_{DESYNC}. Note that T_{DESYNC} will be the true timing error offset that the encoded EMP synchronisation result need to compare with during verification.

In contrast, the relative offset between IMU device 1 and IMU device 2 remains ideally equal to T_{PHASE}, under the assumption that both devices use the same magnetometer model and thus exhibit approximately identical T_{LAG}, effectively cancelling out any added delay between them.

4.3.2 MLX90393 Magnetometer Configuration and Time Lag Analysis

To obtain more accurate values for both T_{DESYNC} and T_{PHASE}, the MLX90393 magnetometer is configured to operate with the shortest possible measurement window (T_{CONV}). This is achieved by selecting the smallest digital filter setting, the lowest ADC oversampling rate, enabling 2-phase Hall plate spinning, and restricting measurements to the x-axis magnetic field only. According to the manufacturer's datasheet, these settings reduce T_{CONV} to 192 μs, which is sufficiently narrow for precise verification of the encoded EMP synchronisation scheme.

Apart from the timing quantities (T_{STBY} and T_{ACTIVE}) shown in **Figure 9**, an additional delay must be considered due to the MLX90393's use of a level-sensitive trigger, which is internally sampled at 250 μs intervals. This introduces an extra delay, T_{TRIG}, of maximum 250 μs between the assertion of the trigger signal and its internal recognition by the device. Consequently, the total time lag T_{LAG} can be calculated by:

$$T_{\mathrm{LAG}} = T_{\mathrm{TRIG}} + T_{\mathrm{STBY}} + T_{\mathrm{ACTIVE}} \tag{5}$$

With these configurations applied, the complete timing sequence of a single measurement cycle in the MLX90393 is illustrated in **Figure 36**. The corresponding timing values and their ranges are summarised in **Table 8**.

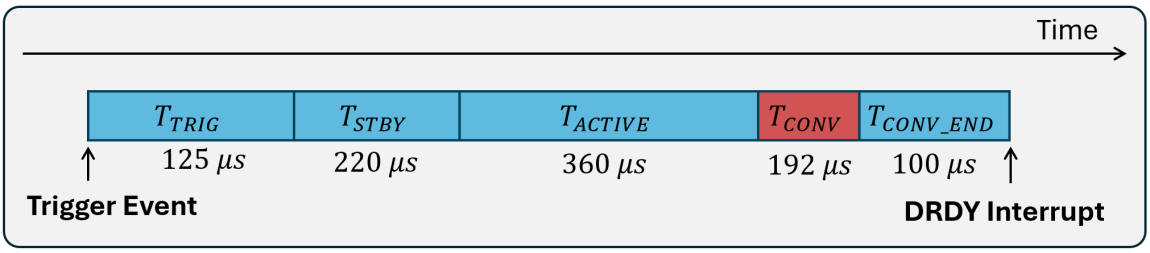


Figure 36: Time sequence of a single measurement event in MLX90393 when initiated by a trigger signal. Note that the time values displayed are all typical values.

Timing Name	Minimum (μs)	Typical (μs)	Maximum (μs)
T_{TRIG}[a]	0.01	125[b]	250
T_{STBY}	176	220	264
T_{ACTIVE}	228	360	432
$T_{\mathrm{CONV\_END}}$	80	100	120
T_{LAG}[c]	404	705	946

Table notes:
a Provided as trigger width in the datasheet.
b Average value is used.
c Calculated according to Equation 5.

Table 8: Different timing values for a single measurement in MLX90393 and their value range. Data are sourced from the official datasheet [30]. Note that these values are added together to obtain T_{LAG}. The possible variation of the T_{LAG} from the typical value will be a limiting factor for the testbed time resolution.

Thus, the T_{LAG} value for MLX90393 has a typical value of 705 μs, with minimum of 404 μs and maximum of 946 μs. Unfortunately, there is currently no effective way to directly measure a precise

value of T_{LAG} for individual chip, thus, in this work, the typical value is used. The difference between the true T_{LAG} value and the typical value will pose a limitation on the resolution of the verification platform design. In the next section, some indirect tests will be set up to investigate the impact of the T_{LAG} variation to the sampling timing of the testbed.

4.3.3 Testing T_{LAG} Variations

To study the impact of T_{LAG} variation to the sampling timing of the testbed, both across successive measurements and between different IMU devices, the same experimental setup and configuration as **Section 4.2.8** is used: $T_{\mathrm{SAMPLE}} = 40\,\mathrm{ms}$, $T_{\mathrm{PHASE}} = 2.75\,\mathrm{ms}$, $T_{\mathrm{OFFSET}} = 40\,\mathrm{ms}$, $w = 320\,\mathrm{ms}$, and $a = 10\,\mathrm{ms}$. The key difference lies in the timestamping strategy: instead of capturing timestamps at trigger assertion, timestamps are now recorded upon the reception of the DRDY interrupt, revealing the effective measurement sequence completion time by taken into account all the timings shown on **Figure 36**.

To evaluate intra-device variation, the time intervals between successive samples, denoted as T'_{SAMPLE}, are examined. Unlike the fixed 40 ms interval used for issuing measurement triggers (as described in **Section4.2.8**), deviations observed in T'_{SAMPLE} indicate variability in the sensor's internal processing time. Since the testbed generates trigger signals at precisely controlled 40 ms intervals, any fluctuation in the recorded sample intervals must result from timing uncertainties within the sensor itself.

To assess inter-device variation, time differences between adjacent samples (recorded as T'_{PHASE}) from magnetometer 1 and magnetometer 2 are compared. Deviations from the expected T_{PHASE} indicate T_{LAG} mismatches of the T_{LAG} value between the devices.

The definations of T'_{SAMPLE} and T'_{PHASE} are summarised in **Table 9**.

Timing Name	Definition
T'_{SAMPLE}	The time intervals between the timestamps of **successive samples from the same magnetometer**, when the testbed timestamping strategy is checking data ready interrupt.
T'_{PHASE}	The time intervals between the timestamps of **adjacent samples from magnetometer 1 and magnetometer 2**, when the testbed timestamping strategy is checking data ready interrupt.

Table 9: Timing Definitions for T'_{SAMPLE} and T'_{PHASE}. Note that Their derivations from T_{SAMPLE} and T_{PHASE} will reveal potential impacts of the variation of T_{LAG}.

The result of these tests and corresponding analysis can be found in **Section 5.3**.

4.4 Design of the Sample Pattern Pre-Processing Program

After the FPGA-based testbed uploads data containing magnetometer samples and synchronisation session metadata, the raw data must be captured and processed by a pre-processing program. This program simplifies the sample patterns for both human inspection and automated analysis. This section first introduces the notation used to encode sample patterns during encoded EMP synchronisation, then details the design of the processing pipeline, and finally presents a test for evaluating the program's accuracy. All scripts used in this stage are available in the Synchronisation Wand project's GitHub repository [42].

4.4.1 The Plus-Minus-Equal Notation

To facilitate interpretation of sample patterns during an encoded EMP synchronisation session, a symbolic notation is introduced. Each electromagnetic pulse is summarised using one of three main symbols: "+", "−" and "=", followed by an optional "*". The meaning of each symbol is as follows:

- **Plus (+)**: Indicates that the current electromagnetic pulse contains one additional magnetometer sample compared to the expected number.
- **Equal (=)**: Indicates that the number of magnetometer samples during the current pulse exactly matches the expected value (i.e., w/s; see **Section 3.1.2** for details).
- **Minus (−)**: Indicates that one sample is missing compared to the expected count. This scenario is theoretically not possible under normal operation and usually suggests an issue during data acquisition.
- **Asterisk (*)**: Appended to any of the above symbols, it indicates that a sample was detected on the leading edge of the pulse. This can occur due to the transient response of the electromagnet. In this system, all edge-aligned samples are attributed to the leading edge of the following pulse.

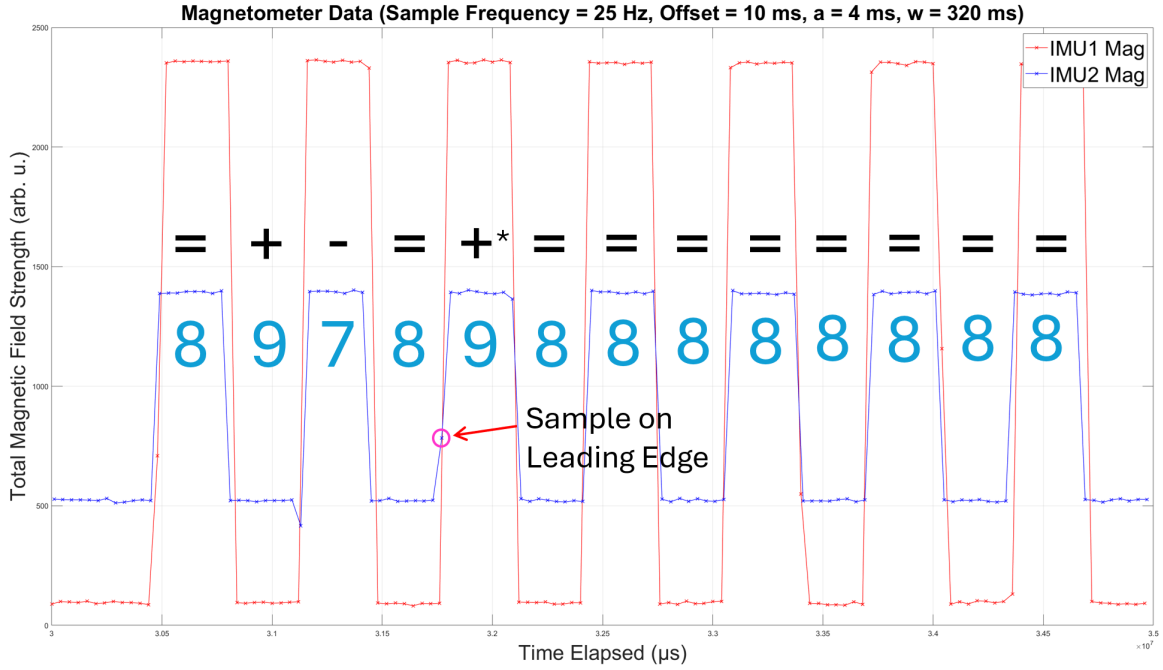


Figure 37: The magnetometer samples collected during an encoded EMP synchronisation session and plotted. The amount of samples of each IMU2 magnetometer pulse is annotated, with their corresponding sign based on the plus-minus-equal notation. Note that pulses with expected amount of samples points (8) are marked with "=", with one extra sample (9) are marked with "+", with one less sample (7) are marked with "-". If the sample lies on the rising edge, an extra "*" will be annotated.

Figure 37 illustrates an example pattern of magnetometer samples recorded during an encoded EMP synchronisation session. The sample counts for each IMU 2 pulse are annotated, along with their corresponding symbols under this notation. The minus sign in this example is caused by a corrupted data point.

4.4.2 The Pattern Pre-processing Pipeline Design

The overview of the pattern pre-processing pipeline is shown in **Figure 38**, and their individual components will be introduced detailedly below:

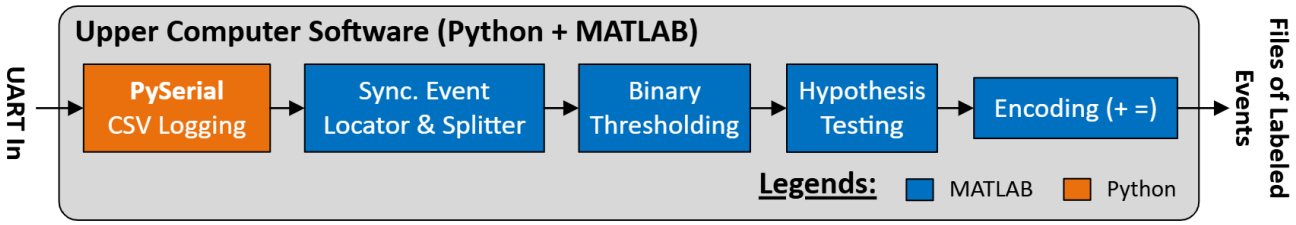


Figure 38: An overview of the sample pattern pre-processing pipeline. Note that sample data from UART serial signal will first be handled and logged in CSV in Python, then it is feed into a pipeline in MATLAB, which first split the individual tests, perform binary thresholding, hypothesis testing and finally encode with the plus-minus-equal notation to output.

CSV Logging:

At this stage, serial UART data is received and decoded into bytes using the *PySerial* library in Python. The script continuously scans the incoming byte stream for the packet header byte (0x2C). Upon detecting a valid header, it reads the following byte to identify the source of the data packet (e.g., 0x01 for IMU device 1, 0x02 for IMU device 2, and 0xFF for the master device), and decodes the payload using the appropriate format (see **Figure 44**).

Sample data from IMU device 1 and IMU device 2 is then written in real-time to a CSV file named `imu_data_log.csv`. Synchronisation session metadata transmitted by the master device and including session separation timestamps and corresponding T_{OFFSET} values is recorded in a separate CSV file, `imu_EM_log.csv`.

Synchronisation Event Locator and Splitter:

This stage uses a MATLAB script to extract individual synchronisation sessions from `imu_data_log.csv`, which may contain hundreds of sessions. Based on the separation timestamps recorded in `imu_EM_log.csv`, the script splits the data into individual CSV files. Each output file is annotated with the corresponding T_{OFFSET} value in its filename for traceability.

Binary Thresholding and Hypothesis Testing:

Binary thresholding and hypothesis testing techniques are applied to correctly classifying the peaks inside the pattern. This process is visualised in **Figure 39**:

The encoded EMP signal is initially segmented using binary thresholding, where the threshold θ is defined as the midpoint between the global maximum and minimum amplitudes of the sampled signal:

$$\theta = \frac{\max(y) + \min(y)}{2} \tag{6}$$

where y denotes the raw sampled signal. This coarse segmentation splits the signal into candidate high/low pulses.

To refine the segmentation, a statistical hypothesis test is applied to the last sample of each candidate pulse, determining whether it belongs to the current pulse or the subsequent rising/falling edge (e.g.,

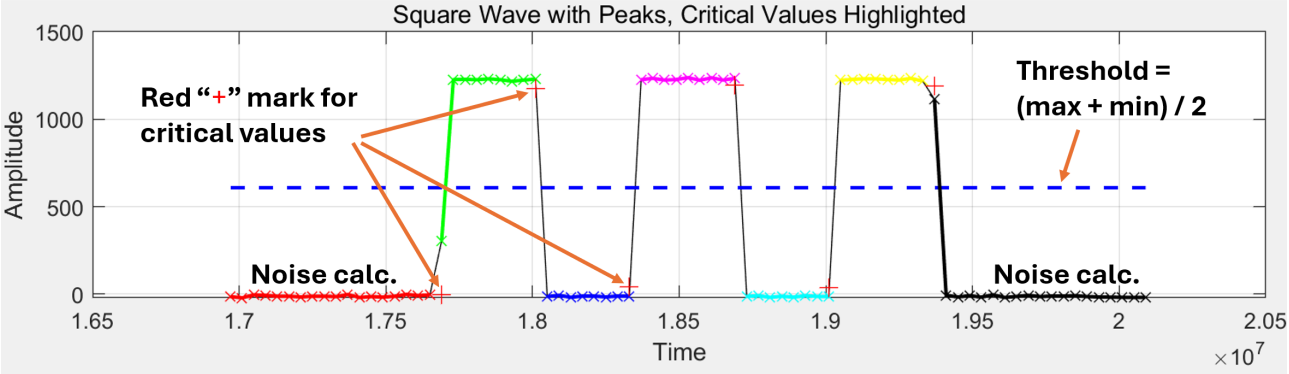


Figure 39: A visualisation of how binary thresholding and hypothesis testing work to classify the pulses for a given pattern. The classified pulses are highlighted with different colours for easier distinguishing. The red plus markers are used to annotate the critical value of the hypothesis test, and the blue dashed line is used to annotate the threshold.

the sample on the first rising edge in **Figure 39**, which belongs to the initial pulse but not being classified correctly by the binary thresholding). The test leverages the following assumptions:

1. **Noise Model**: Amplitude variance within pulses arises from additive white Gaussian noise (AWGN) with zero mean and stationary variance σ^2.
2. **Population Variance (σ^2)**: Estimated from background noise samples at the head and tail of the signal (non-pulse regions). These two regions are annotated as "Noise calc." in **Figure 39**.
3. **Population Mean (μ)**: Computed for each pulse by excluding the first and last two samples.

For a high pulse, a lower-tail test is performed on the last sample y_{last} with null hypothesis:

$$H_0 : y_{\text{last}} \geq \mu - z_\alpha \cdot \sigma \tag{7}$$

where z_α is the critical value for standardised normal distribution at significance level α. The significance level used for this pre-processing pipeline is $\alpha = 10^{-6}$. If y_{last} falls below the critical value $\mu - z_\alpha \cdot \sigma$, it is reclassified as the leading edge of the subsequent low pulse. Conversely, for a low pulse, an upper-tail test ($H_0 : y_{\text{last}} \leq \mu + z_\alpha \cdot \sigma$) is applied, with reclassification triggered if y_{last} exceeds the critical value $\mu + z_\alpha \cdot \sigma$. These critical values are annotated in **Figure 39** with red "+" markers.

After these two tests, all pulses can be correctly classified, and the next step is to encode the pattern using the plus-minus-equal notation.

Encoding:

Based on the previously classified pulses, plus-minus-equal encoding is performed by counting the number of sample points within each pulse and comparing it against the expected count, defined as w/s. To determine whether a pulse includes a leading-edge sample, a hypothesis test similar to the one used earlier is applied to the first sample value y_{first} of each classified pulse. Unlike before, no reclassification is performed; instead, if the null hypothesis H_0 is rejected, it is interpreted as evidence that the first sample lies on the leading edge, and an asterisk (*) is appended to the corresponding plus, minus, or equal symbol.

The encoded results are stored in individual CSV files for each synchronisation session, with the associated T_{OFFSET} value annotated in the filename for traceability. This encoding enables downstream scripts to easily locate the index of the extra-sample pulse (p) by identifying the position of the first plus symbol in the sequence.

4.4.3 Program Accuracy Verification

The accuracy of the pre-processing program is verified by manually inspecting its output after encoding and comparing it against the original sample patterns (e.g., visualisations similar to **Figure 39**). If a clear error is observed, such as an extra pulse generated due to a corrupted sample, the case is marked as failed. In this evaluation, 100 consecutive encoded EMP synchronisation test cycles were executed by the testbed and analysed using the program, and the accuracy of the program is estimated based on the binary test results using the Clopper-Pearson method (See **Section 3.4**). The verification results are shown in **Section 5.4**.

4.5 Tests and Setup for the Encoded EMP Synchronisation Method Verification

With the testbed and the pre-processing script ready, the systematically verification of the encoded EMP synchronsiation method can be carried out. Two different binary tests (Test 1 and Test 2) are used to verify the encoded EMP synchronisation method, which emulates two different application scenarios:

4.5.1 Test 1: Synchronise One IMU Device with the Master Device

This test is designed to emulate an application scenario, when the user want to synchronise the RTC time on an IMU device to align with a reference accurate real-time on the synchronisation master device, which sends the synchronisation pulse (e.g., the synchronisation wand). In this case, the timing error is T_{DESYNC} (See **Figure 35**). To pass the test, the encoded EMP synchronisation should predict a time adjustment value equal to T_{DESYNC}, with a permitted absolute error no greater than the precision a (See **Section 3.1.2**). Representing mathematically, test 1 will be marked as passed if:

$$|p_1 a - T_{\text{DESYNC}}| \leq a \tag{8}$$

where p_1 is the extra-sample-pulse-index of IMU device 1.

4.5.2 Test 2: Synchronise Two IMU Devices Together

This test is designed to emulate another application scenario, when the user want to synchronise the RTC time on two different IMU devices to align with each other by performing one single encoded EMP synchronisation together. In this scenario, the timing error is T_{PHASE} (See **Figure 22**). To pass the test, the encoded EMP synchronisation should predict a time adjustment value equal to T_{PHASE}, with a permitted absolute error no greater than the precision a (See **Section 3.1.2**).

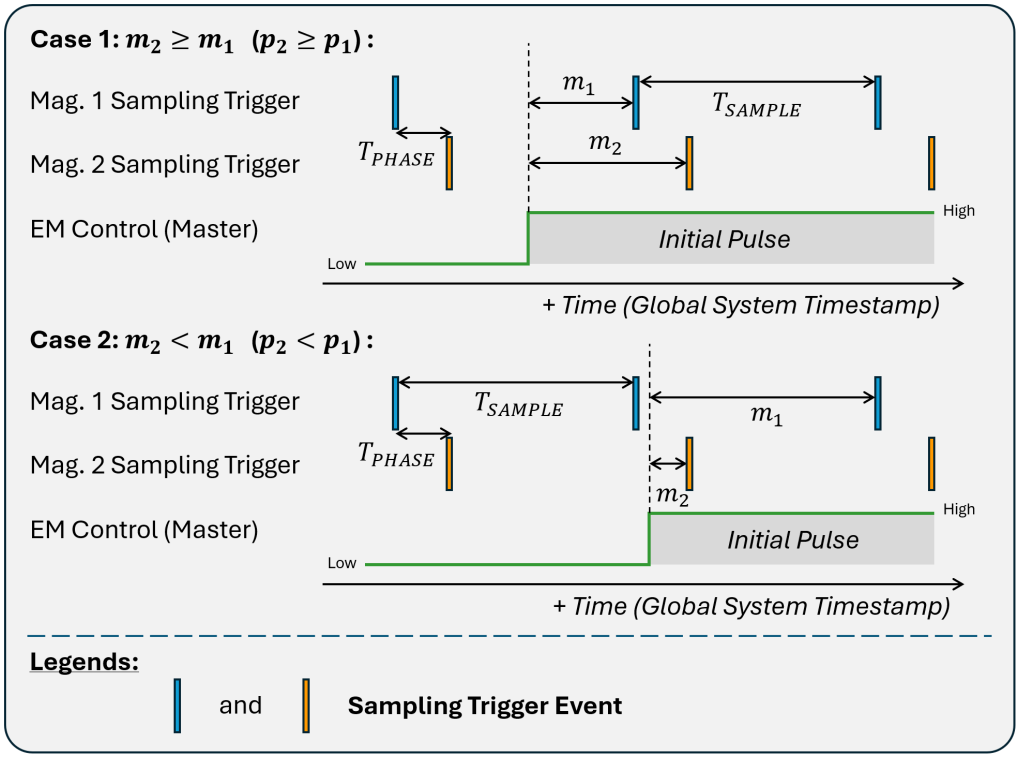


Figure 40: The timing diagram showing two possible cases for test 2 which need separate treatments. Note the different position of the EM control signal. In the first case, $m_2 \geq m_1$, and in the second case, $m_2 < m_1$.

For this scenario, depending on when the synchronisation EM pulse signal is sent (i.e. different T_{OFFSET}, but its exact value is not necessary to be known for analysis), there exists two different scenarios, with the timing diagrams shown in **Figure 40**.

Case 1 is the most direct case, where $m_2 \geq m_1$. This will also result in a larger or equal extra-sample-pulse index on IMU device 2 (p_2) than / to p_1. In this case, the time adjustment value predicted by the encoded EMP synchronisation event is given by $(p_2 - p_1)a$. By comparing with the reference T_{PHASE} value, this leads to a pass condition of:

$$|(p_2 - p_1)\, a - T_{\text{PHASE}}| \leq a \tag{9}$$

Case 2 will cause $m_2 < m_1$. This case will also lead to a $p_2 < p_1$ as well. In this case, the time adjustment value predicted by the encoded EMP synchronisation event is given by $(p_2 - p_1)a + T_{\text{SAMPLE}}$. By comparing with the reference T_{PHASE} value, this leads to a pass condition of:

$$|(p_2 - p_1)a + T_{\text{SAMPLE}} - T_{\text{PHASE}}| \leq a \tag{10}$$

4.5.3 Experiment Setup

To set up the experiment, two MLX90393 magnetometer breakout board are placed within the active range (< 5 cm) of the electromagnet (The same one which is being used in [10]). The experimental layout is shown in **Figure 41** below:

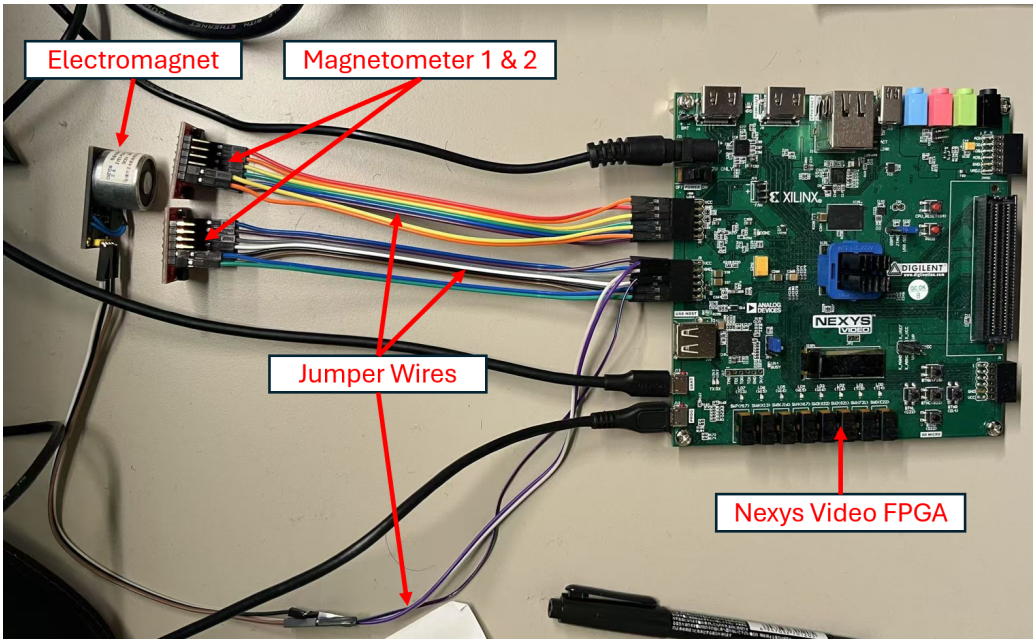


Figure 41: A photo of the experiment layout. The MLX90393 magnetometer chips are placed within the active range of the electromagnet ($\leq 5\,cm$). All IMUs and the electromagnet module are connected to the Nexys Video FPGA board using jumper wires.

Some common parameters that will not change during the experiment are configures as follows: $w = 320\,ms$, $T_{\text{SAMPLE}} = 40\,ms$, $T_{\text{PHASE}} = 2.75\,ms$.

Five groups of experiments are performed, with the max synchronisation error (a) setting from least accurate to most accurate: a = 10 ms, 5 ms, 2 ms, 1 ms and 0.5 ms.

Inside each experiment group, 400 synchronisation events will be performed. Across these 400 tests, T_{OFFSET} will be adjusted from 0 ms to 40 ms with steps of 0.1 ms. This act is to cover the full

possibe range of T_{OFFSET} (since T_{SAMPLE} is set to 40 ms), simulating the real world consideration when performing encoded EMP synchronisation - the user may start performing synchronisation at any random time.

The result of this verification and corresponding analysis can be found in **Section 5.5**

4.6 Additional Test for Magnetometer Without Direct Access

An additional simple test is carried out to study a special case: how magnetometers without direct host access (e.g., AK09916 in ICM20948; see **Section 3.3.4**) affect the encoded EMP synchronization accuracy. To do this, the testbed's internal logic was modified to interface with the ICM20948 IMU. Due to its indirect magnetometer access architecture, the workflow requires three stages:

1. Configure AK09916 in continuous measurement mode at 100 Hz.
2. Program the ICM20948's auxiliary I^2C master to poll magnetometer registers at 1.1 kHz (~5× Nyquist rate) and cache data in shared sensor registers.
3. Sample the shared register bank at 25 Hz ($T_{\mathrm{SAMPLE}} = 40$ ms) with timestamping upon acquisition.

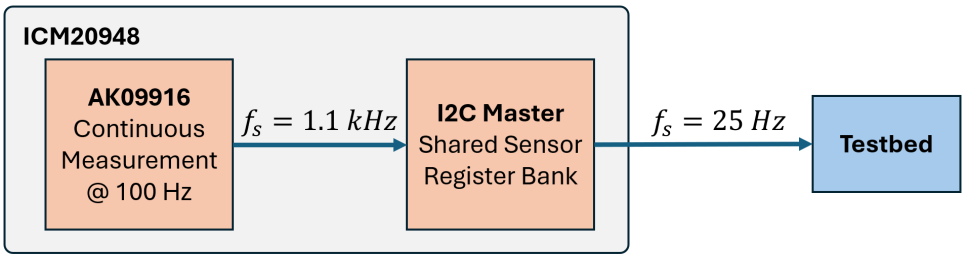


Figure 42: The data flow from the AK09916 magnetometer to the testbed. Note that the AK09916 samples at 100 Hz, the sample data from AK09916 is first being transmitted to the shared sensor register bank by the I2C master at a sampling frequency at 1.1 kHz (much higher above the nyquist frequency of 200 Hz), and the testbed samples the data from the shared sensor register bank at 25 Hz.

Figure 42 illustrates this data flow. Unlike directly accessible magnetometers (e.g., MLX90393), the ICM20948's design precludes single-measurement triggering or data-ready interrupts, making T_{DESYNC} unobservable. Consequently, Test 1 (See **Section 4.5**) cannot be executed. However, T_{PHASE} remains controllable by initializing Magnetometer 2's measurement sequence T_{PHASE} after Magnetometer 1, enabling Test 2 analysis.

Four experimental groups were defined and carried out with the parameters in **Table 10**.

Experiment Group Index	T_{SAMPLE} (ms)	T_{PHASE} (ms)	a (ms)	w (ms)	Number of tests
1	40	10	8	320	4
2	40	10	6	320	4
3	40	10	4	320	5
4	40	20	2	400	5

Table 10: A table showing the configurations of four experiment groups, with the maximum synchronisation error (a) configuration ranging from the least accurate (8 ms) to most accurate (2 ms)

5 Results, Analysis and Discussion

5.1 BLE Functionality

The BLE functionalities, including its connectivity, signal strength, and firmware functionalities are verified by performing the test in **Section 4.1.2**. The results are described as followed:

100 out of 100 pairing attempts at both 1 m and 10 m range are success. This gives a success rate of at least 96.38% by estimation using the Clopper-Pearson method, which meets the criteria of $\geq 95\%$ specified in the project goal.

The screenshot of the scatter plot received signal strength indicator (RSSI) versus time is shown in **Figure 43**. It can be easily observed that the average RSSI level is above the reference line of -75 dBm. This meet the requirement of average RSSI $\geq$ -80 dBm, showing good BLE signal strength.

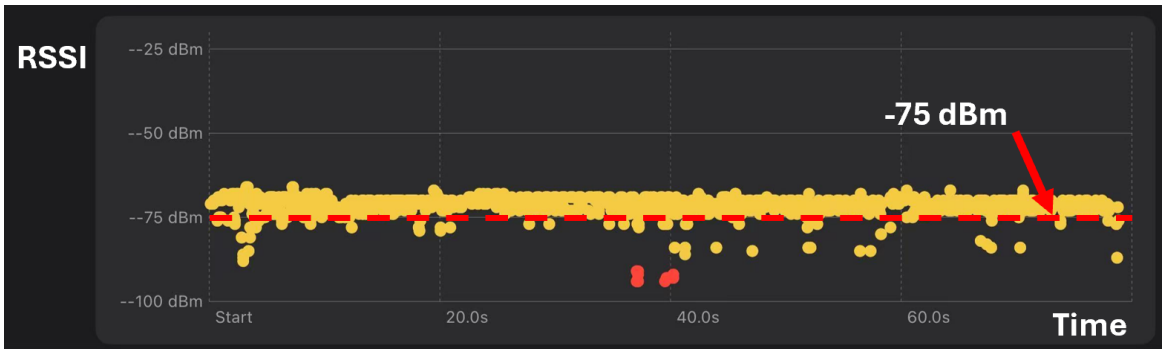


Figure 43: The BLE received signal strength indicator (RSSI) of the synchronisation wand at a separation of 10 meters without obstacle. The red and yellow scatters are measured data points. Note that the majority of samples are above the reference line of -75 dBm.

For the parameter configuration stress test, 100 out of 100 attempts are successful, corresponding to a success rate of at least 96.38%, and no system abort message observed on the serial monitor. This shows high reliability and robustness of the BLE firmware design, which meet the requirement specified in the project goal (See **Section 2.2**).

5.2 Testbed Functionalities

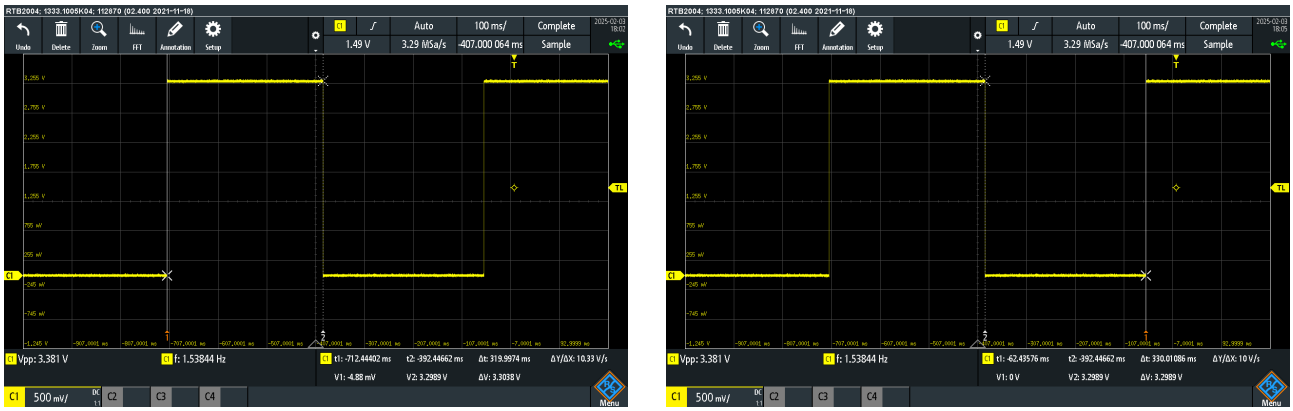
By following the procedure in **Section 4.2.8**, the results of testbed functionality verification are presented and discussed below:

Electromagnet Control Signal Accuracy:

Figure 44a and **Figure 44b** show the time cursor measurements of the initial pulse width and the first successive pulse, respectively. The initial pulse duration is measured as 319.9974 ms, while the first successive pulse is 330.01086 ms. Compared with the expected pulse widths (320 ms and 330 ms), the error calculated are 0.0008125%, and 0.00329%, both below the maximum value (0.01%) requirement specified in the project goal. These results confirm that the testbed generates accurate electromagnet control signals.

Sampling Trigger Timing Accuracy:

Figure 45 shows the plotted distribution of T_{SAMPLE} for the IMU device 1 and IMU device 2, and the T_{PHASE} between them. Statistical parameters show that T_{SAMPLE} values are precisely at 40 ms, and T_{PHASE} at 2.75 ms, both with zero standard variations. This confirms the high accuracy of the sampling trigger timing implemented in the testbed.



(a) (b)

Figure 44: Time cursor measurement result of: (a) the initial pulse for 319.9974 ms, (b) the 1st pulse for 330.01086 ms. The expected values are 320 ms and 330 ms separately.

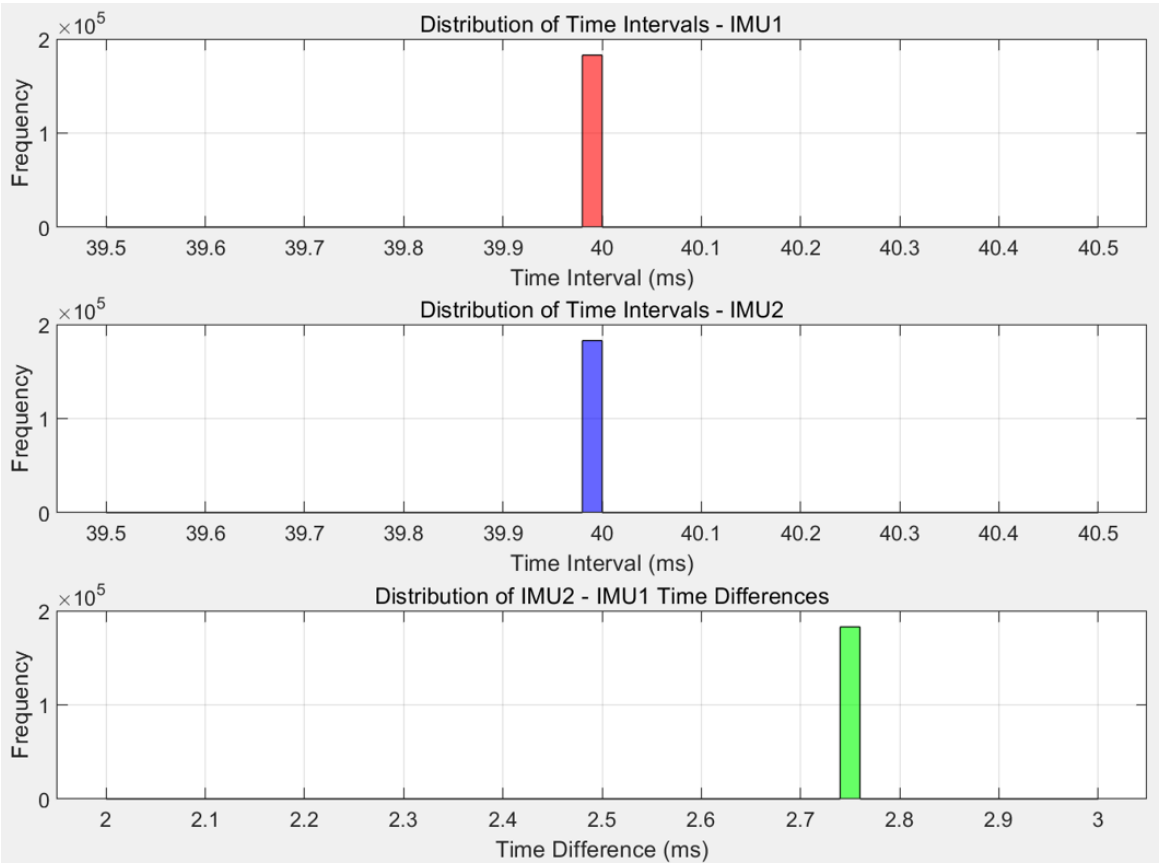


Figure 45: Distribution of T_{SAMPLE} for the two emulated IMU devices and T_{PHASE} across them. Note that there are no derivation in these values, demonstrating highly accurate timing of the sampling triggers asserted by the testbed.

Automated Testing Function Reliability:

The reliability of the automated testing function in the testbed is validated by analysing the scatter plot of T_{OFFSET} values recorded in the received synchronisation metadata packets (**Figure 46**). As shown, T_{OFFSET} increases linearly from 0ms to 40ms across the first 400 tests before being reset to 0 ms to avoid overflow, repeating the cycle thereafter. No data points are missing or exhibit abnormal values. This result confirms the correct operation of the automated testing mechanism

and demonstrates its high reliability in executing over 100 consecutive synchronisation experiments, meeting the requirements specified in the project goals (See **Section 2.2**).

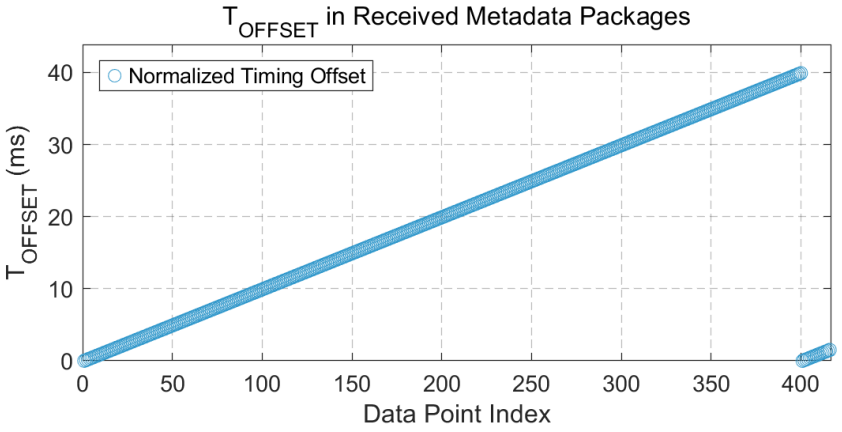


Figure 46: Variation of T_{OFFSET} across 400+ automated synchronisation tests. Note that it increases linearly from 0 ms to 40 ms, and being reset back to 0 ms and re-increase. No missing or abnormal data observed, indicating high stability of the system.

5.3 T_{LAG} Variations - A Limitation of the Proposed Verification Method

The T_{LAG} variation is studied by performing the test specified in **Section 4.3.3**. **Figure 47** shows the plotted distribution of T'_{SAMPLE} for the IMU device 1 and IMU device 2, and the T'_{PHASE} between them, after changing the testbed to timestamp the DRDY reception event (See **Table 9** for definitions). The calculated statistical parameters are recorded in **Table 11**.

Name	Mean (ms)	Standard Derivation (ms)	Minimum (ms)	Maximum (ms)
T'_{SAMPLE} for IMU device 1	40	0.008586	39.979714	40.020214
T'_{SAMPLE} for IMU device 2	40	0.008034	39.980607	40.019500
T'_{PHASE}	2.718610	0.008233	2.698786	2.738607

Table 11: Statistical parameters of the distributions of two T'_{SAMPLE} from different emulated IMU devices and the T'_{PHASE}. Note that the mean of two T'_{SAMPLE} are consistent with pre-set T_{SAMPLE} (40 ms). T'_{PHASE} shows a difference with T_{PHASE} (2.75 ms), revealing that T_{LAG} is different from the typical value and have mismatch between parts.

The results indicate that T_{LAG} exhibits only minor variation between successive measurements. The mean value of T'_{SAMPLE} aligns precisely with the preconfigured T_{SAMPLE}, suggesting that this variation has a negligible impact on the sampling period of the emulated IMU devices.

However, a discrepancy of approximately 320 μs is observed between the measured mean value of T'_{PHASE} and the expected T_{PHASE} of 2.75ms. This reveals that, in practice, the actual T_{LAG} values of the two magnetometer chips deviate from the assumed typical value of 705μs and differ from each other. Since the verification platform design in this work assumes a uniform T_{LAG} equal to the typical value, this mismatch introduces a quantifiable inaccuracy, with a worst-case offset of up to 320 μs, which defines the resolution of the verification platform.

This forms the primary limitation of the testbed design, potentially slightly reducing verification accuracy when testing encoded EMP synchronisation at very fine maximum synchronisation error settings (e.g., $a = 0.5$ ms). Nonetheless, the method remains sufficiently accurate for verifying synchronisation performance at $a \geq 1$ms, thereby fulfilling the 0.5 ms maximum timing error requirement defined in the project goals.

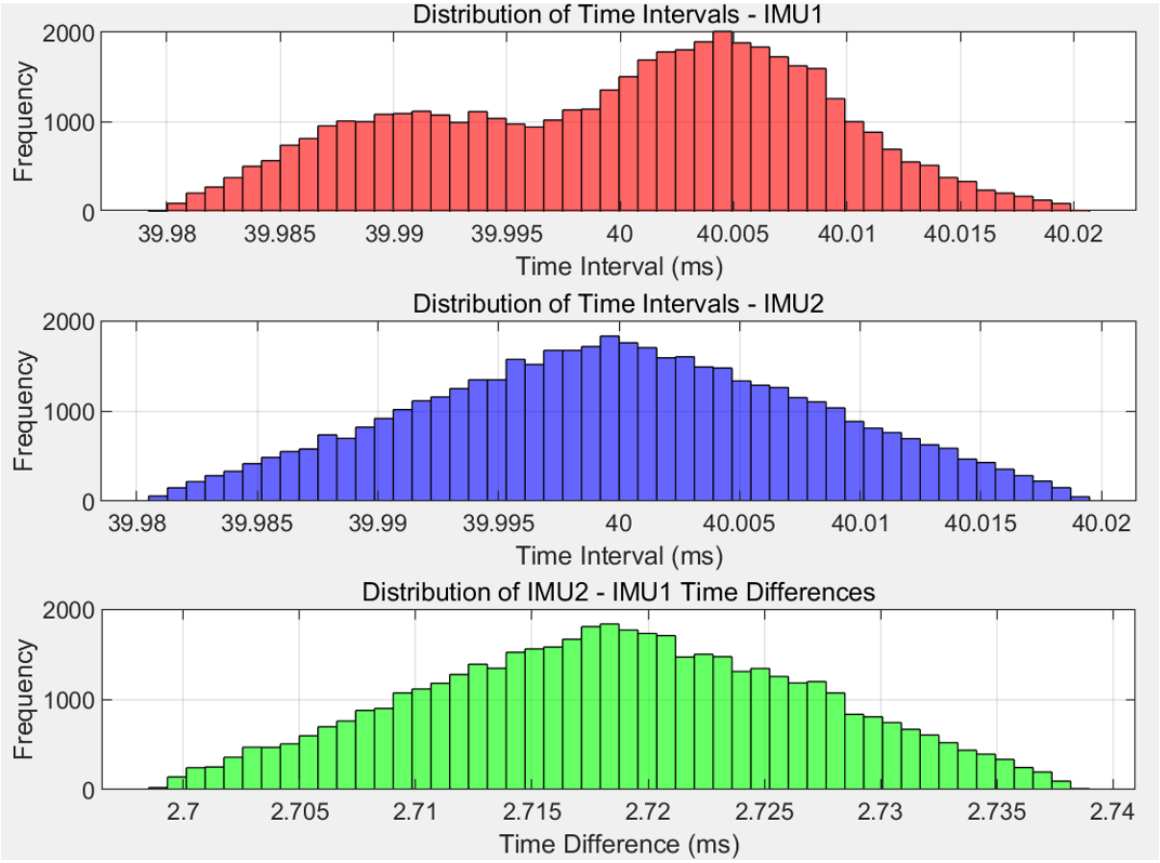


Figure 47: Distribution of T'_{SAMPLE} for the two emulated IMU devices and T'_{PHASE} across them when timestamping event is switched to the DRDY reception. Note that the distribution is no longer a single bin but split across the time axis, revealing T_{LAG} variations between measurements.

5.4 Accuracy of the Pre-processing Program

By applying the verification test specified in **Section 4.4.3**, the accuracy of the pre-processing program can be assessed. 100 out of 100 tests passed, where no obvious error is found in the result of the pre-processing program. By applying the Clopper-Pearson method at 95% confidence level, it can be estimated that the accuracy of the program has at least 96.38%, meeting the accuracy requirement specified by the project goal.

5.5 Accuracy of the Encoded EMP Synchronisation Method

5.5.1 Experiment Results

After performing the verification tests described in **Section 4.5**, the pass rate of both test 1 and test 2 for the five different experiment groups are shown in **Table 12**:

It can be seen that all five different max synchronisation error (a) settings show high pass rate above 95 %, and for $a \geq 1\,ms$, almost all the tests are passed. This demonstrated high robustness and reliability of the expanded EMP synchronisation method. Additionally, by using this testbed and verification methodology, it is successfully to proved that the expanded emp synchronisation method can reliably achieve an accuracy of 0.5 ms, which is 6 times more accurate than the previously verified value of 3 ms [10]. This reaches the level of accuracy required for many high precision applications like 1 ms for gait analysis [14].

Max Synchronisation Error (a) Setting	Test 1 Pass Rate	Test 2 Pass Rate
10 ms	99.76 %	100.00 %
5 ms	100.00 %	100.00 %
2 ms	99.50 %	100.00 %
1 ms	100.00 %	100.00 %
0.5 ms	96.03 %	95.04 %

Table 12: Table summarising the pass rate of test 1 and test 2 for five different max synchronisation error settings from least accurate to most accurate. Note that all configurations achieved high pass rate above 95 %.

5.5.2 Explanation of the Lower-than-Average Pass Rate for a = 0.5 ms

The reduced pass rate observed for the $a = 0.5\,\text{ms}$ configuration can be attributed to inaccuracies in the assumed magnetometer T_{LAG}. During verification, a nominal value of 705 μs was assigned to both devices. However, experimental results revealed an inter-device T_{LAG} mismatch of approximately 320 μs, indicating that the actual lag times deviate from the assumed value (See **Section 5.3**). This discrepancy introduces slight errors in the calculated values of both T_{DESYNC} and T_{PHASE}. While such errors are negligible when the max synchronisation error setting (a) is large (e.g., $a \geq 1\,\text{ms}$), they become significant when a approaches the same order of magnitude as the lag variation—such as in the $a = 0.5\,\text{ms}$ case. This reveals the bottle neck of the testbed design, and future works are required for improvement.

This variation in T_{LAG} also highlights a practical limitation in the achievable accuracy of the encoded EMP synchronisation method when performing to commercial IMU devices, which is the part-to-part timing inconsistency.

5.6 Synchronisation Performance for Magnetometer Without Direct Access

5.6.1 Experiment Results

To briefly investigate the performance of encoded EMP synchronisation method for magnetometers without direct access, a small number of tests specified in **Section 4.6** are performed, and their results are shown in **Tables 13 to 16** below:

Sync. Event	p_1	p_2	Sync. Error (ms)	Result
No. 1	3	5	-6	Passed
No. 2	4	5	2	Passed
No. 3	3	4	2	Passed
No. 4	2	3	2	Passed

Table 13: Verification 1. $T_{\text{SAMPLE}} = 40$ ms, $T_{\text{OFFSET}} = 10$ ms, $a = 8$ ms, $w = 320$ ms.

Sync. Event	p_1	p_2	Sync. Error (ms)	Result
No. 1	4	5	4	Passed
No. 2	1	3	-2	Passed
No. 3	2	4	-2	Passed
No. 4	3	5	-2	Passed

Table 14: Verification 2. $T_{\text{SAMPLE}} = 40$ ms, $T_{\text{OFFSET}} = 10$ ms, $a = 6$ ms, $w = 320$ ms.

Sync. Event	p_1	p_2	Sync. Error (ms)	Result
No. 1	4	7	-2	Passed
No. 2	5	8	-2	Passed
No. 3	?	?	?	Unknown[a]
No. 4	?	?	?	Unknown[a]
No. 5	6	7	6	Failed[b]

Table notes:
a Unknown due to unexpected behaviour.
b Failed because the synchronisation error lies outside the accept range (4 ms).

Table 15: Verification 3. $T_{\text{SAMPLE}} = 40$ ms, $T_{\text{OFFSET}} = 10$ ms, $a = 4$ ms, $w = 320$ ms. Note that more than half of the tests does not pass, due to unexpected behaviour or failed to meet the pass conditions.

Sync. Event	p_1	p_2	Sync. Error (ms)	Result
No. 1	2	9	6	Failed[a]
No. 2	?	?	?	Unknown[b]
No. 3	15	4	-2	Passed
No. 4	2	11	2	Passed
No. 5	17	6	-2	Passed

Table notes:
a Failed because the synchronisation error lies outside the accept range (4 ms).
b Unknown due to unexpected behaviour.

Table 16: Verification 4. $T_{\text{SAMPLE}} = 40$ ms, $T_{\text{OFFSET}} = 20$ ms, $a = 2$ ms, $w = 400$ ms. Note that almost half of the tests does not pass, due to unexpected behaviour or failed to meet the pass conditions.

All 8 tests for $a = 8\,ms$ and $a = 6\,ms$ settings passes. This indicates that the encoded EMP synchronisation still performs reliably on large maximum synchronisation error (a) configuration for magnetometers with indirect access. However, for small a settings, unexpected behaviours and failed tests start to become common, which contributes to nearly half of total (See **Table 15**) and **Table 16**.

5.6.2 Analysis of the Failed and Unusually Behaved Synchronisation Sessions

To analyse the failed and unusually behaved synchronisation sessions mentioned in **Section 5.6.1**, their patterns are listed in **Table 44** below using the plus-minus-equal notation:

It can be noticed that, the failed events contains many "$-$" sign, which is impossible according to the theory of the encoded EMP synchronisation method (See **Section 3.1.2**). Also, the presence of "$-$" sign is not observed when carrying out the experiments using the MLX90393 chips. This concludes that having indirect access to the magnetometer can significantly degrade the encoded EMP synchronisation precision, and it is not recommended to perform the encoded EMP synchronisation on this kind of devices.

Event	IMU	Init.	1st	2nd	3rd	4th	5th	6th	7th	8th	9th	10th
Ver. 3, No. 3	1	−	+*	=	=	=	=	=	=	=	=*	=*
Ver. 3, No. 3	2	+*	−	=*	+*	=	=	=	=	=	=	=*
Ver. 3, No. 4	1	+*	=	=	=	=	=	=	=	=	=*	=
Ver. 3, No. 4	2	=	+[a]	−	=	+*	=	=	=	=	=	=
Ver. 3, No. 5	1	=	=	=	=	=*	=	+*	=	=	=	=
Ver. 3, No. 5	2	=	=	=	=	=	=	=	+	−	+*	=
Ver. 4, No. 1	1	−	=*	+*	=	=	=	=	=	=	=	=
Ver. 4, No. 1	2	=	=	=*	=	=	=	=	=	=	+	=
Ver. 4, No. 2	1	+	−	=*	=	=	=*	+*	=	=	=	=
Ver. 4, No. 2	2	=	=	=	=	=	=	=	=	=	=	=

Continued...

Event	11th	12th	13th	14th	15th	16th	17th	18th	19th	20th	21st	22nd
Ver. 3, No. 3	+*	=	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 3, No. 3	=	=*	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 3, No. 4	+*	=	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 3, No. 4	=	=	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 3, No. 5	=	=	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 3, No. 5	=	=	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Ver. 4, No. 1	=	=	=	=	=	=*	=	=	=	+*	=	=
Ver. 4, No. 1	=	=	=	=	=	=	=	=	=	=	=	=
Ver. 4, No. 2	=	=	=	=	=	=	=	=	=	=	=*	=
Ver. 4, No. 2	=	=	=	+	−	+*	=	=	=	=	=	=

Table notes:

a Sign of data corruption on the last sample.

Table 17: Patterns of synchronisation events which are wrong or with unexpected behaviours. The "N/A"s means that the current synchronisation session does not contain this pulse. Note that many "−" sign are observed, an some tests even shows multiple "+" within a synchronisation session, which are theoretically impossible. This indicates a potential timing issue due to indirect access.

6 Conclusions and Future Work

6.1 Key Achievements

This project delivers two pivotal advancements in the field of wearable sensor synchronisation. The first is the development of the Synchronisation Wand, a portable embedded system designed to achieve sub-millisecond alignment between wearable devices through encoded electromagnetic pulses (EMP) detected via magnetometers. Building on earlier published work, the Wand was extended with a Bluetooth Low Energy (BLE) configuration interface, which demonstrated robust performance with a pairing and command success rate of 96.38%, a strong signal level (RSSI $>$ –80 dBm at 10 m line-of-sight), and 100% crash-free operation across 100 trials.

The second major contribution is a comprehensive verification platform that sets a new standard for systematic evaluation of synchronisation methods in wearable systems. This platform integrates a fully automated FPGA-based testbed and a dedicated data processing program. The testbed achieved sub-microsecond precision in sampling trigger control, EMP pulse generation with $<$0.01% pulse width

error, and a timing resolution of 320 μs—providing sufficient granularity to reliably verify synchronisation methods with accuracy down to 0.5 ms. It supported over 400 fully autonomous test cycles with 0% crash rate and no manual intervention. The accompanying data pre-processing pipeline successfully extracted synchronisation patterns from raw magnetometer signals with 96.38% classification accuracy.

Using this platform, the encoded EMP synchronisation method was verified to consistently achieve a maximum synchronisation error of 0.5 ms—representing a sixfold improvement over previously reported values. This level of precision meets the typical requirements for high-accuracy applications such as gait analysis (1 ms). However, the evaluation also identified a critical limitation: the encoded EMP synchronisation method is unsuitable for systems using magnetometers without direct host access, due to the presence of uncontrollable internal measurement delays.

6.2 Limitations and Future Work

The most significant current limitation of the verification platform lies in hardware-level variation in the internal measurement timing of magnetometer chips. Deviations from the typical internal time lag, as well as mismatches between different chips, introduce systematic errors in timing analysis and reduce the effective resolution of the testbed. To address this, future work could explore two directions: developing methods to measure and compensate for time lag variation in real time—such as onboard monitoring and auto-correction mechanisms—or redesign the testbed to replace the magnetometer with an analogue Hall-effect sensor. The latter option would eliminate internal sampling latency altogether, offering a fundamental solution to this limitation and significantly improving the temporal resolution of the verification platform.

Further extensions of this work may include implementing mechanisms to emulate real-time clock (RTC) drift, in order to evaluate the effectiveness of repeated synchronisation in mitigating long-term timing divergence. In addition, improving synchronisation accuracy in systems with indirect access to magnetometer data remains an open challenge. One potential direction is the use of learning-based methods, such as neural network models to analyse the pattern and predict the correct extra-sample-pulse-index.

References

[1] M. Abbas and R. Le Bouquin Jeannès, "Acceleration-based gait analysis for frailty assessment in older adults," *Pattern Recognition Letters*, vol. 161, pp. 45–51, 2022.

[2] J. Yang, T.-H. Huang, S. Yu, X. Yang, H. Su, A. M. Spungen, and C.-Y. Tsai, "Machine learning based adaptive gait phase estimation using inertial measurement sensors," vol. 2019 Design of Medical Devices Conference of *Frontiers in Biomedical Devices*, p. V001T09A010, 04 2019.

[3] Y. Sun, D. A. Greaves, G. Orgs, A. F. de C. Hamilton, S. Day, and J. A. Ward, "Using wearable sensors to measure interpersonal synchrony in actors and audience members during a live theatre performance," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 7, Mar. 2023.

[4] N. Gao, W. Shao, M. S. Rahaman, and F. D. Salim, "n-gage: Predicting in-class emotional, behavioural and cognitive engagement in the wild," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 4, Sept. 2020.

[5] T. Plotz, C. Chen, N. Y. Hammerla, and G. D. Abowd, "Automatic synchronization of wearable sensors and video-cameras for ground truth annotation – a practical approach," in *2012 16th International Symposium on Wearable Computers*, pp. 100–103, 2012.

[6] A. Spilz and M. Munz, "Synchronisation of wearable inertial measurement units based on magnetometer data," *Biomedical Engineering / Biomedizinische Technik*, vol. 68, no. 3, pp. 263–273, 2023.

[7] C. Wang, Z. Sarsenbayeva, C. Luo, J. Goncalves, and V. Kostakos, "Improving wearable sensor data quality using context markers," in *Adjunct Proceedings of the 2019 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2019 ACM International Symposium on Wearable Computers*, UbiComp/ISWC '19 Adjunct, (New York, NY, USA), p. 598–601, Association for Computing Machinery, 2019.

[8] J. A. Ward, G. Pirkl, P. Hevesi, and P. Lukowicz, "Detecting physical collaborations in a group task using body-worn microphones and accelerometers," in *2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, pp. 268–273, 2017.

[9] A. Hoelzemann, H. Odoemelem, and K. Van Laerhoven, "Using an in-ear wearable to annotate activity data across multiple inertial sensors," in *Proceedings of the 1st International Workshop on Earable Computing*, EarComp'19, (New York, NY, USA), p. 14–19, Association for Computing Machinery, 2020.

[10] T. J. Gilbert, Z. Lin, S. Day, A. F. d. C. Hamilton, and J. A. Ward, "A magnetometer-based method for in-situ syncing of wearable inertial measurement units," *Frontiers in Computer Science*, vol. 6, 2024.

[11] MOKOBlue, "Understanding the measures of bluetooth rssi," *MOKOBlue Blog*, 2022.

[12] Google LLC, "Fast Pair Certification Guideline." [Online]. Available: `https://developers.google.com/nearby/fast-pair/fast-pair-certification-guideline`, 2024. [Accessed: 8-April-2025].

[13] Y. Han, T. J. Gilbert, X. Tan, and J. A. Ward, "The wand chooses the imu - open source hardware for synchronising wearables using magnetometers," in *Companion of the 2024 on ACM International Joint Conference on Pervasive and Ubiquitous Computing*, UbiComp '24, (New York, NY, USA), p. 939–943, Association for Computing Machinery, 2024.

[14] V. Masalskyi, D. Čičiurėnas, A. Dzedzickis, U. Prentice, G. Braziulis, and V. Bučinskas, "Synchronization of separate sensors' data transferred through a local wi-fi network: A use case of human-gait monitoring," *Future Internet*, vol. 16, no. 2, 2024.

[15] N. Semiconductors, "Mcx c series: Energy efficient and cost effective mcus," *NXP Smarter World Blog*, 2024. Accessed: 2025-04-09.

[16] Espressif Systems, "ESP32-S3 Series Datasheet." [Online]. Available: `https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf`, 2024. [Accessed: 1-April-2025].

[17] Abracon LLC, "ATXK-H11 32.768kHz TCXO Datasheet." [Online]. Available: `https://abracon.com/datasheets/ATXK-H11.pdf`, 2024. [Accessed: 1-April-2025].

[18] Cisco Systems, "Use Best Practices for Network Time Protocol." [Online]. Available: `https://www.cisco.com/c/en/us/support/docs/availability/high-availability/19643-ntpm.html`, 2024. [Accessed: 9-April-2025].

[19] H. W. Johnson and M. Graham, "High-speed digital design: A handbook of black magic," 1993.

[20] Compaq Computer Corporation and Hewlett-Packard Company and Intel Corporation and Lucent Technologies Inc and Microsoft Corporation and NEC Corporation and Koninklijke Philips Electronics N.V., "Universal Serial Bus Specification Revision 2.0." [Online]. Available: `https://www.usb.org/document-library/usb-20-specification`, April 2000. [Accessed: 3-April-2025].

[21] Espressif Systems, "ESP-IDF Programming Guide." [Online]. Available: `https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html`, 2025. [Accessed: 4-April-2025].

[22] FreeRTOS, "FreeRTOS Documentation Overview." [Online]. Available: `https://www.freertos.org/Documentation/00-Overview`, 2025. [Accessed: 4-April-2025].

[23] Bluetooth Special Interest Group (SIG), "Bluetooth Core Specification Version 6.0." [Online]. Available: `https://www.bluetooth.com/specifications/specs/core-specification-6-0/`, August 2024. [Accessed: 4-April-2025].

[24] Nordic Semiconductor, "Ultra Low Power Wireless Solutions: Bluetooth Low Energy Power Consumption." [Online]. Available: `https://www.nordicsemi.com/Products/Low-power-short-range-wireless/Bluetooth-Low-Energy`, 2023. [Accessed: 9-April-2025].

[25] Bluetooth Framework, "Connection Parameters in Bluetooth Low Energy." [Online]. Available: `https://www.btframework.com/connparams.htm`, 2024. [Accessed: 9-April-2025].

[26] Nordic Semiconductor, "Bluetooth Low Energy Fundamentals." [Online]. Available: `https://academy.nordicsemi.com/courses/bluetooth-low-energy-fundamentals/`, 2025. [Accessed: 3-April-2025].

[27] K. A. Clark, Z. Zhou, and Z. Liu, "Clock synchronizing radio access networks to picosecond precision using optical clock distribution and clock phase caching," *J. Opt. Commun. Netw.*, vol. 16, pp. A89–A97, Jan 2024.

[28] Digilent, "Nexys video artix-7 fpga: Trainer board for multimedia applications," 2025. Accessed: 2025-04-09.

[29] H. Heidari, *Current-Mode High Sensitivity CMOS Hall Magnetic Sensors*. PhD thesis, 01 2015.

[30] Melexis, "MLX90393 Datasheet." [Online]. Available: https://www.melexis.com/en/documents/documentation/datasheets/datasheet-mlx90393, September 2024. [Accessed: 4-April-2025].

[31] STMicroelectronics, "LIS3MDL Datasheet." [Online]. Available: https://www.st.com/resource/en/datasheet/lis3mdl.pdf, December 2023. [Accessed: 4-April-2025].

[32] MEMSIC Semiconductor Co., Ltd., "MMC5603NJ Datasheet." [Online]. Available: https://www.memsic.com/Public/Uploads/uploadfile/files/20220119/MMC5603NJDatasheetRev.B.pdf, January 2022. [Accessed: 4-April-2025].

[33] Asahi Kasei Microdevices, "AK09912 3-Axis Electronic Compass Datasheet." [Online]. Available: https://www.digikey.co.uk/htmldatasheets/production/1746545/0/0/1/AK09912.pdf, July 2014. [Accessed: 4-April-2025].

[34] TDK InvenSense, "ICM-20948 9-Axis MEMS MotionTracking™ Device Datasheet, Revision 1.3." [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2016/06/DS-000189-ICM-20948-v1.3.pdf, 2017. [Accessed: 4-April-2025].

[35] Wikipedia contributors, "Serial Peripheral Interface." [Online]. Available: https://en.wikipedia.org/wiki/Serial_Peripheral_Interface, 2025. [Accessed: 1-April-2025].

[36] Rohde Schwarz, "Understanding UART." [Online]. Available: https://www.rohde-schwarz.com/uk/products/test-and-measurement/essentials-test-equipment/digital-oscilloscopes/understanding-uart_254524.html, 2025. [Accessed: 1-April-2025].

[37] Stack Overflow user, "Valid-Ready Handshake in Verilog." [Online]. Available: https://stackoverflow.com/questions/53583946/valid-ready-handshake-in-verilog, 2018. [Accessed: 4-April-2025].

[38] K. Clark, "Elec0028 note - timing and synchronisation in digital hardware," 2025.

[39] University of Illinois at Urbana-Champaign, "CS 341 Coursebook: Scheduling." [Online]. Available: https://cs341.cs.illinois.edu/coursebook/Scheduling, 2024. [Accessed: 4-April-2025].

[40] Brown, Lawrence D. and Cai, T. Tony and DasGupta, Anirban, "Interval Estimation for a Binomial Proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001. [Online; accessed 8-April-2025].

[41] W. Q. Meeker and L. A. Escobar, *Statistical Methods for Reliability Data*. New York: John Wiley & Sons, 1998.

[42] Yuxuan Han, "Synchronisation-Wand." [Online]. Available: https://github.com/YuxuanHan0326/Synchronisation-Wand, 2025. [Accessed: 31-March-2025].

Appendices

A Appendix

This appendix only shows a few extractions for the key verilog code extraction of some modules. For more complete version of these, or for the verilog implementation of other modules, please navigate to the GitHub repository of the Synchronisation Wand [42].

1.1 Key Verilog Code Extraction of the Timestamp Generator

This is the verilog code extraction of how the timestamp generator is implemented.

```
always @(posedge i_clk or negedge i_reset) begin
    if (~i_reset) begin
        r_timestamp <= 40'd0;
        clk_divider_counter <= 0;
    end
    else begin
        if (clk_divider_counter == CLK_DIVIDER - 1) begin
            clk_divider_counter <= 0;
            r_timestamp <= r_timestamp + 1;  // 2nd counter for timestamp
        end
        else begin
            clk_divider_counter <= clk_divider_counter + 1;  // 1st counter
        end
    end
end
```

1.2 Key Verilog Code Extraction of the SPI master

1.2.1 Handshake and SPI Clock Generation

This is the verilog code extraction of the handshake mechanism and SPI clock generation is implemented.

```
// When data valid present
if (i_TX_DV) begin
  o_TX_Ready      <= 1'b0;  // Set ready low
  r_SPI_Clk_Edges <= 16;  // Total # edges in one byte ALWAYS 16
end

// Generate SPI CLK
else if (r_SPI_Clk_Edges > 0) begin
  o_TX_Ready <= 1'b0;
  if (r_SPI_Clk_Count == CLKS_PER_HALF_BIT*2-1) begin
    r_SPI_Clk_Edges <= r_SPI_Clk_Edges - 1'b1;
    r_Trailing_Edge <= 1'b1;
    r_SPI_Clk_Count <= 0;
    r_SPI_Clk       <= ~r_SPI_Clk;  // Reverse Clock
  end
  else if (r_SPI_Clk_Count == CLKS_PER_HALF_BIT-1) begin
    r_SPI_Clk_Edges <= r_SPI_Clk_Edges - 1'b1;
    r_Leading_Edge  <= 1'b1;
    r_SPI_Clk_Count <= r_SPI_Clk_Count + 1'b1;
    r_SPI_Clk       <= ~r_SPI_Clk;  // Reverse Clock
  end
  else begin
```

```
    r_SPI_Clk_Count <= r_SPI_Clk_Count + 1'b1;
  end
end

// Transmission Finished
else begin
  o_TX_Ready <= 1'b1;  // Set ready high
end
```

1.2.2 Transmission and Reception

For transmission:

```
// If ready is high, reset bit counts to default
if (o_TX_Ready)
begin
  r_TX_Bit_Count <= 3'b111;
end
// Transmit first bit
else if (r_TX_DV & ~w_CPHA) begin
  o_SPI_MOSI     <= r_TX_Byte[3'b111];
  r_TX_Bit_Count <= 3'b110;
end
// Transmit following bits
else if ((r_Leading_Edge & w_CPHA) | (r_Trailing_Edge & ~w_CPHA)) begin
  r_TX_Bit_Count <= r_TX_Bit_Count - 1'b1;
  o_SPI_MOSI     <= r_TX_Byte[r_TX_Bit_Count];
end
```

For reception:

```
// If ready is high, reset bit counts to default
if (o_TX_Ready) begin
  r_RX_Bit_Count <= 3'b111;
end
// Receive Data
else if ((r_Leading_Edge & ~w_CPHA) | (r_Trailing_Edge & w_CPHA)) begin
  o_RX_Byte[r_RX_Bit_Count] <= i_SPI_MISO;  // Sample data
  r_RX_Bit_Count            <= r_RX_Bit_Count - 1'b1;
  // Pulse data valid if RX finished
  if (r_RX_Bit_Count == 3'b000)
  begin
    o_RX_DV   <= 1'b1;   // Byte done, pulse Data Valid
  end
end
```

1.3 The Look Up Table

This is the implementation of the look up table module. This look up table module is already pre-filled with the register values needed for MLX90393 initialisation.

```
    module lut_256 (
    input wire [7:0] addr,     // 8-bit address for 256 locations
    output reg [7:0] data      // 1-byte data output
);

    // Define the lookup table as a register array
    reg [7:0] lut [0:255]; // 256 words, each 8 bits wide
```

```
8
9       initial begin
10          // Write Group 1, exit current mode
11          lut[0]   = 8'h80;
12          lut[1]   = 8'h00;
13
14          // Write Group 2, Reset, total 2 bytes
15          lut[2]   = 8'hF0;
16          lut[3]   = 8'h00;
17
18          // Write Group 1, Initialisation, total 15 bytes
19          // 0x00
20          lut[4]   = 8'h60;   // head
21          lut[5]   = 8'h00;   // upper byte
22          lut[6]   = 8'h70;   // lower byte
23          lut[7]   = 8'h00;   // address << 2
24          lut[8]   = 8'h00;   // Read Status
25
26          lut[9]   = 8'h60;   // head
27          lut[10]   = 8'h48;   // upper byte
28          lut[11]   = 8'h80;   // lower byte
29          lut[12]   = 8'h04;   // address << 2
30          lut[13]   = 8'h00;   // Read Status
31
32          lut[14]  = 8'h60;   // head
33          lut[15]  = 8'h00;   // upper byte
34          lut[16]  = 8'h00;   // lower byte
35          lut[17]  = 8'h08;   // address << 2
36          lut[18]  = 8'h00;   // Read Status
37
38          // write Group 2, measurement_1, total 2 bytes
39          lut[19]  = 8'h32;   // SM, x only
40          lut[20]  = 8'h00;
41
42          // write Group 3, measurement_2, total 4 bytes
43          lut[21]  = 8'h42;   // RD, x only
44          lut[22]  = 8'h00;   // Read status
45          lut[23]  = 8'h00;   // Read x_upper
46          lut[24]  = 8'h00;   // Read x_lower
47
48          // Reserved
49          lut[25]  = 8'h00;   // Reserved
50          lut[26]  = 8'h00;   // Reserved
51          lut[27]  = 8'h00;   // Reserved
52          lut[28]  = 8'h00;   // Reserved
53          lut[29]  = 8'h00;   // Reserved
54
55          // ...
56          lut[255] = 8'h00; // Example last value
57      end
58
59      // Read data from the lookup table based on the address
60      always @(*) begin
61          data = lut[addr];
62      end
63
64  endmodule
```

1.4 The Delay Timer

This is the verilog code extraction of how a timer is implemented inside the sample data acquisition and timestamping finite state machine.

```
if (~timer_IR_flg) begin
    if (delay_counter == timer_CC1_value) begin
        timer_CC1_flg <= 1;  // Set CC1 flag
    end else begin
        timer_CC1_flg <= 0;  // Reset CC1 flag
    end
    if (delay_counter < timer_autoreload_value) begin
        delay_counter <= delay_counter + 1;
    end else begin
        timer_IR_flg <= 1;        // Set timer flag
        delay_counter <= 32'd0; // Reset delay counter
    end
end
```